

ТЕХНИЧЕСКАЯ ДОКУМЕНТАЦИЯ

Warehouse

Система межскладских перевозок на Django и SQLAlchemy

Архитектура, модели данных, бизнес-процессы, безопасность,
эксплуатация и развитие

Состояние кода: ветка `main`, коммит `9f08ae7` (25.09.2026)

Редакция книги: 27.09.2026

О книге

Книга описывает проект Warehouse файл за файлом: что делает каждый модуль, почему он устроен именно так, как связан с остальными и какие были альтернативы. Все факты взяты из исходного кода и истории git. Тесты проекта прогнаны при подготовке: 14 тестов веб-слоя и 6 тестов BLL проходят.

Условные обозначения: ✓ — реализовано; **заглушка** — объявлено, но пока показывает «в разработке»; блоки с полосой слева — замечания, найденные проблемы и рекомендации.

Диаграммы построены на Mermaid; их исходники лежат в `documentation/Warehouse_Technical_Book.md`.

Состав книги: 1–2 — обзор и структура; 3–8 — Django-часть и модели; 9–12 — API, сервисы, БД, файлы; 13–18 — тесты, конфигурация, безопасность, производительность, логи, развёртывание; 19–21 — зависимости, архитектура, развитие; приложения А–С — глоссарий, FAQ, ссылки.

Оглавление

1 Введение и обзор проекта	6
1.1 Назначение проекта и решаемые бизнес-задачи	6
1.2 Краткая история развития проекта	6
1.3 Технологический стек и обоснование выбора	7
1.4 Архитектурный обзор	8
1.5 Основные возможности системы	10
2 Структура проекта	11
2.1 Полное дерево файлов	11
2.2 Корневые файлы	12
2.3 Организация приложений и их взаимодействие	13
2.4 Структура шаблонов и статических файлов	14
2.5 Файлы конфигурации и окружения	14
3 Модели данных	15
3.1 Где живут модели	15
3.2 ER-диаграмма	15
3.3 Справочники (entities/reference.py)	18
3.4 Warehouse (entities/warehouse.py)	19
3.5 Employee (entities/employee.py)	19
3.6 Product и StockOnWarehouse	20
3.7 Перевозки (entities/shipment.py)	20
3.8 OperationHistory (entities/operation_history.py)	21
3.9 Методы моделей, менеджеры и QuerySet	22
3.10 DTO — «модели» для остальных слоёв	22
3.11 Миграции: история и структура	23
4 Представления (Views)	24
4.1 Обзор и принцип «тонких» view	24
4.2 Class-based или function-based views	24
4.3 Страницы аутентификации (web/views/auth.py)	25
4.4 Заглушки (_stub.py) и будущие страницы	26
4.5 JSON API (web/views/api.py)	26
4.6 Декораторы, применяемые к view	28
4.7 Пагинация, фильтрация, сортировка	28
5 URL-маршрутизация	29
5.1 Полная карта URL	29
5.2 Вложенность и принципы именования	30
5.3 Именованные URL в шаблонах и коде	31
5.4 API endpoints	31
6 Шаблоны (Templates)	33
6.1 Наследование	33
6.2 web/base.html	33
6.3 Остальные шаблоны	33
6.4 Custom template tags и filters (templatetags/warehouse_tags.py)	34
6.5 Context processors	35
6.6 Стили (static/web/app.css) и frontend	35
7 Формы (Forms)	36

7.1 RegisterForm	36
7.2 Разделение валидации	36
7.3 Widgets	37
7.4 Formsets	37
7.5 Загрузка файлов (web/uploads.py)	37
8 Аутентификация и авторизация	39
8.1 Система пользователей	39
8.2 Механизм аутентификации	39
8.3 Permissions и проверка склада (AccessService)	41
8.4 Декораторы проверки прав	42
8.5 Session management	42
9 Django REST Framework	43
9.1 Статус	43
9.2 Сериализация на DTO	43
9.3 Когда стоит перейти на DRF или Django Ninja	43
10 Бизнес-логика и сервисный слой	45
10.1 Состав BLL	45
10.2 Жизненный цикл перевозки	46
10.3 Процесс «перевозка через транзитный склад» целиком	46
10.4 ShipmentDraftService — черновик	47
10.5 ShipmentDispatchService — резерв, отправка, отмена	48
10.6 ShipmentReceiptService — приёмка	49
10.7 ShipmentTransitCoordinator — cross-docking	49
10.8 RouteQueryService — чтение	50
10.9 EmployeeService	50
10.10 Общий код: SenderGuardsMixin	50
10.11 Signals, management commands, фоновые задачи	50
11 Работа с базой данных	52
11.1 Выбор СУБД	52
11.2 Подключение (dal/database.py)	52
11.3 Unit of Work	52
11.4 Репозитории	53
11.5 Оптимизация запросов: аналоги select_related / prefetch_related	54
11.6 Индексы	54
11.7 Транзакции и атомарность	55
11.8 Кэширование запросов и database routers	55
11.9 Схема и начальные данные	55
12 Статические файлы и медиа	57
12.1 Настройки	57
12.2 Сборка и раздача статики	57
12.3 Пользовательские загрузки	57
12.4 Frontend-сборщики	58
13 Тестирование	59
13.1 Структура	59
13.2 Тесты веб-слоя (web/tests.py)	59
13.3 Unit-тесты BLL	60
13.4 Чего не хватает	60
13.5 CI/CD	61

14 Конфигурация и окружение	62
14.1 Переменные окружения	62
14.2 Разбор env_secret	62
14.3 Окружения dev / staging / prod	62
14.4 Best practices, которые уже соблюдены	63
14.5 Замечания	63
15 Безопасность	64
15.1 Реализованные меры	64
15.2 Найденные слабые места и рекомендации	64
15.3 Аудит безопасности	65
16 Производительность и оптимизация	66
16.1 Узкие места	66
16.2 Кэширование	66
16.3 Оптимизация запросов	66
16.4 Асинхронность и CDN	67
17 Логирование и мониторинг	68
17.1 Конфигурация (common/logger.py)	68
17.2 Что и на каком уровне логируется	68
17.3 Sentry, ELK, метрики	69
18 Развёртывание и DevOps	70
18.1 Текущее состояние	70
18.2 Docker и docker-compose (рекомендуемая схема)	70
18.3 CI/CD	71
18.4 Миграции в production	71
18.5 Бэкап и восстановление	71
18.6 Масштабирование	71
19 Зависимости и требования	72
19.1 Пакеты	72
19.2 Версии Python и совместимость	72
19.3 Разделение зависимостей	73
20 Архитектурные решения и паттерны	74
20.1 Применённые паттерны	74
20.2 Почему Django без Django ORM	74
20.3 Trade-offs	75
20.4 Зависимости между пакетами	75
21 Расширение и развитие проекта	77
21.1 Как добавить новую функцию: сквозной пример	77
21.2 Точки расширения	77
21.3 Рекомендации по рефакторингу (по приоритету)	78
21.4 Roadmap (из прототипа и заглушек)	78
Приложение А. Глоссарий	79
Приложение В. FAQ и Troubleshooting	80
Приложение С. Ссылки и ресурсы	81

1 Введение и обзор проекта

1.1 Назначение проекта и решаемые бизнес-задачи

Warehouse — внутренняя информационная система сети складов. Её предмет — **межскладская перевозка товара** (в коде — *shipment*, «перевозка» или «отгрузка»). Перевозка идёт по **маршруту** из двух и более складов. Каждый отрезок маршрута «склад А → склад В» называется **этапом** (*stage*, в интерфейсе — «путь»).

Система отвечает на пять практических вопросов:

1. **Что лежит на складе и сколько из этого свободно?** Остаток (`quantity`) и резерв (`reserved_quantity`) хранятся отдельно. Свободный остаток — их разность (`StockItemDTO.available`).
2. **Кто и что везёт?** Старший кладовщик собирает черновик: маршрут, товары первого этапа, водителя и сопроводительные документы.
3. **Нельзя отдать один и тот же товар дважды.** Перед отправкой товар резервируется. Строки остатков при этом блокируются `SELECT ... FOR UPDATE`, поэтому два кладовщика не могут одновременно зарезервировать одно и то же.
4. **Что приехало на самом деле?** На складе назначения вносят фактическое количество по каждой позиции. Если факт расходится с документом, нужен комментарий, а этап получает статус «Принято с расхождениями».
5. **Что происходит на транзитном складе?** После приёмки промежуточного этапа принятый факт автоматически становится документом следующего этапа и резервируется на транзитном складе. Этим занимается `ShipmentTransitCoordinator`.

Пользователи — сотрудники складов с разными ролями: старший кладовщик, кладовщик, менеджер, водитель и администратор. Права задаются моделью RBAC (*Role-Based Access Control*): права выдаются роли, а не конкретному человеку.

1.2 Краткая история развития проекта

Историю можно восстановить по git-журналу (187 коммитов с 08.09.2026 по 25.09.2026). Выделяются шесть фаз.

Период	Фаза	Ключевые коммиты
08.09	Каркас репозитория	<code>Initial commit</code> , <code>add license</code> , <code>add pytest settings</code> , <code>rename API->WEB</code>
09.09–10.09	Прототип бизнес-логики на чистом Python	<code>feat: Introduce ShipmentStatus Enum</code> , <code>Implement ShipmentDispatchService</code> , <code>Implement ShipmentTransitCoordinator for cross-docking</code> , <code>Implement ShipmentReceiptService</code> , <code>add unittest</code>
14.09–16.09	Слоистая архитектура и SQLAlchemy	<code>add SQLAlchemy</code> , <code>make layers</code> , <code>create repo</code> , <code>Full DAL</code> , <code>add Mappers</code> , <code>add logger</code> , <code>add Auth/Shipment/Dependencies in API</code> (слой FastAPI), <code>add erd.script</code>
17.09	Большая «чистка» и слияние веток	Переход на <code>snake_case</code> и нижний регистр каталогов, слияние веток <code>login+accessService</code> , <code>Transit</code> , <code>QueryService</code> , <code>techlogger</code> , <code>AuditTrail</code>

Период	Фаза	Ключевые коммиты
21.09–24.09	Уточнение предметной области и переход на Django	Разделение dispatch на два сервиса и <code>SenderGuardsMixin</code> , <code>django setup folders</code> , роли «Водитель» и «Менеджер», документы этапа, HTML-прототип интерфейса, связка Django ↔ SQLAlchemy через middleware, <code>operation_history</code>
25.09	«Этап 0» веб-интерфейса	Подготовленная почва для работы с django + <code>login(jwt + bcrypt)</code> , слияние ветки <code>templates</code> , <code>manage.py</code> с автосозданием <code>.env</code>

Наблюдение. Проект начинался как API на **FastAPI**. От того периода остались `common/schemas/*` (модели Pydantic), `common/mappers/auth_mappers.py`, `common/mappers/shipments_statistics.py` и зависимости `fastapi/uvicorn` в `requirements.txt`. Django с ними не работает. Это «археологический слой», его стоит убрать (подробнее в главе 21).

1.3 Технологический стек и обоснование выбора

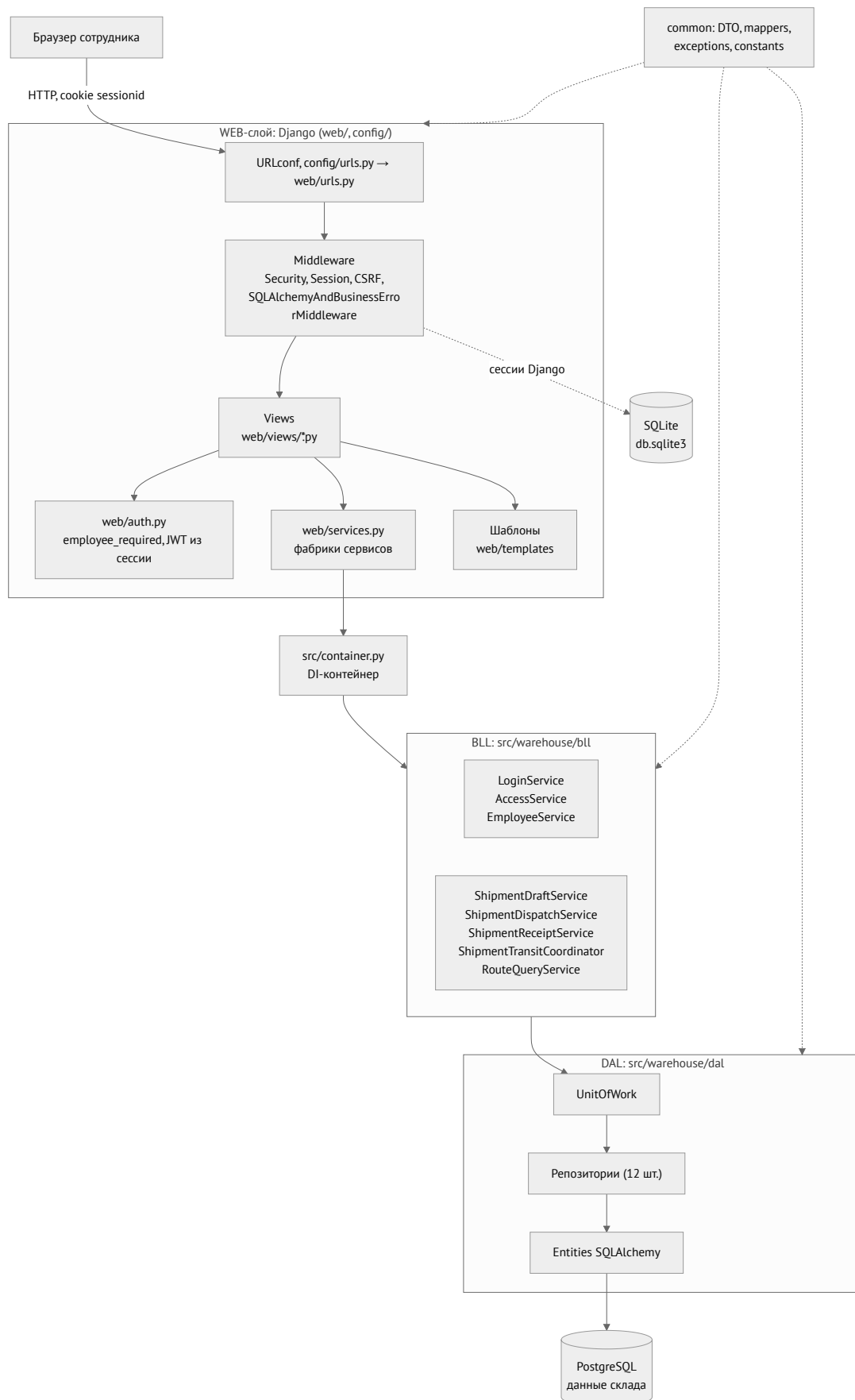
Компонент	Версия	Роль	Почему выбран
Python	≥ 3.12 (требуется Django 6.1)	Язык	<code>StrEnum</code> (3.11+), аннотации <code>X None</code> , <code>dataclass(slots=True)</code>
Django	6.1.1	Веб-слой: маршруты, шаблоны, формы, сессии, CSRF	Зрелый фреймворк со встроенной защитой CSRF/XSS/clickjacking и системой шаблонов
SQLAlchemy	2.0.52	ORM (<i>Object-Relational Mapping</i>) и доступ к данным складского домена	Типизированные модели <code>Mapped[]</code> , <code>with_for_update</code> , PostgreSQL-специфичный <code>INSERT ... ON CONFLICT</code> , полный контроль над транзакциями
psycopg2-binary	2.9.13	Драйвер PostgreSQL	Стандартный драйвер для SQLAlchemy
PostgreSQL	—	Основная СУБД склада	Транзакции, <code>FOR UPDATE</code> , <code>CHECK</code> , <code>numeric(12,3)</code> , <code>jsonb</code>
SQLite	встроен	База самого Django (только сессии)	Не требует настройки, для сессий этого достаточно
PyJWT	2.10.1	Подписанные токены входа (JWT, <i>JSON Web Token</i>)	Лёгкая и распространённая реализация HS256
bcrypt	4.2.1	Хэширование паролей	Медленная адаптивная хэш-функция с солью
dependency-injector	4.45.0	IoC-контейнер (<i>Inversion of Control</i>)	Декларативный граф зависимостей, подмена провайдеров в тестах (<code>override</code>)
python-dotenv	1.2.2	Загрузка <code>.env</code>	Настройки по методологии 12-factor
fastapi, pydantic, uvicorn	0.115 / 2.10 / 0.34	Не используются в текущей версии	Наследие первой версии API

Необычное решение стека — **Django без Django ORM**. Модели и запросы склада написаны на SQLAlchemy. Django берёт на себя HTTP, шаблоны, формы, сессии и защиту. Почему так сложилось и чем это оборачивается, разобрано в главе 20.

1.4 Архитектурный обзор

Код разделён на три слоя и общий модуль `common` :

- **WEB** (`web/`, `config/`) — Django: принимает HTTP, проверяет сессию, разбирает формы и вызывает сервисы.
- **BLL** (*Business Logic Layer*, `src/warehouse/bll/`) — сервисы с бизнес-правилами. Каждый публичный метод сам открывает транзакцию.
- **DAL** (*Data Access Layer*, `src/warehouse/dal/`) — модели SQLAlchemy, репозитории и Unit of Work.
- **common** (`src/warehouse/common/`) — DTO, мапперы, исключения, константы, логирование, безопасность.



Через границы слоёв передаются только **DTO** (*Data Transfer Object*) — замороженные `dataclass`. Модели SQLAlchemy наружу из репозиториев не выходят. Поэтому шаблоны и view не могут случайно вызвать ленивую загрузку (*lazy load*) после закрытия сессии.

1.5 Основные возможности системы

Реализовано полностью, в BLL и DAL:

- вход по логину и паролю: JWT с временем жизни 8 часов, пароли в bcrypt;
- регистрация сотрудников администратором;
- RBAC: 5 ролей, 6 прав и проверка «свой склад» для каждого действия;
- черновик перевозки: маршрут из N складов, товары, водитель, документы;
- резерв с блокировкой строк и отправка со списанием остатка;
- отмена перевозки до отправки с возвратом резерва;
- приёмка с вводом факта, фиксацией расхождений и оприходованием;
- транзит (*cross-docking*): перенос факта в следующий этап и его резервирование;
- просмотр маршрутов с учётом роли: всё, свой склад или только свои рейсы.

Реализовано в веб-слое (на 25.09.2026):

- страницы входа, выхода и регистрации сотрудника;
- JSON API: вход, остатки, создание черновика, отправка, приёмка и отмена;
- шаблон страниц, стили, фильтры статусов и количеств, страницы 403 и 404;
- загрузка файлов документов (`web/uploads.py` ; вызывается пока только из будущих view).

Заглушки «в разработке»: главная, остатки, список перевозок, черновик, карточка перевозки, приёмка. View для них уже объявлены и подключены к URL, но вызывают `stub_page` / `stub_action`. Целевой вид экранов показывает `documentation/prototype.html`.

2 Структура проекта

2.1 Полное дерево файлов

```
Warehouse/
├── .env.example          # шаблон переменных окружения (копируется в .env)
├── .gitignore            # исключения git: venv, .env, db.sqlite3, media/, staticfiles/
├── LICENSE.md           # лицензия
├── README.md            # заголовок «# Warehouse», пока без содержания
├── manage.py            # точка входа Django + автосоздание .env + src в sys.path
├── requirements.txt      # зависимости Python
├── sql_fullset.txt       # эталонный DDL PostgreSQL + начальные данные
├── erd.script           # DDL, сгенерированный ERD-инструментом pgAdmin
├── db.sqlite3           # база сессий Django (в .gitignore)
├── logs/warehouse.log   # файл логов (RotatingFileHandler)
├── documentation/
│   ├── Untitled.png     # ER-диаграмма (dbdiagram.io)
│   └── prototype.html    # кликабельный HTML-прототип интерфейса
├── config/              # «проект» Django
│   ├── __init__.py
│   ├── settings.py      # настройки; читает .env
│   ├── urls.py          # корневой URLconf → include('web.urls')
│   ├── wsgi.py          # точка входа WSGI-сервера
│   └── asgi.py          # точка входа ASGI-сервера
├── web/                 # единственное Django-приложение
│   ├── apps.py          # WebConfig.ready() → setup_logging()
│   ├── auth.py          # sign_in/sign_out, authenticate, employee_required
│   ├── context.py       # context processor current_user
│   ├── errors.py        # BusinessError → 404/403/messages
│   ├── forms.py         # RegisterForm
│   ├── middleware.py    # закрытие сессии SQLAlchemy, JSON-ошибки API
│   ├── models.py        # пусто: Django-моделей нет
│   ├── migrations/      # пусто
│   ├── services.py      # DI-контейнер и фабрики сервисов для view
│   ├── uploads.py       # сохранение и удаление файлов документов
│   ├── urls.py          # 25 URL-маршрутов
│   ├── tests.py         # 14 тестов веб-слоя
│   ├── templatetags/warehouse_tags.py # status_class, status_label, qty, status_pill
│   ├── views/           # auth, home, shipments, draft, detail, receipt, api, _stub
│   ├── templates/       # base, stub, auth/login, auth/register, 403, 404
│   └── static/web/app.css # вся стилистика интерфейса
├── src/
│   ├── main.py          # CLI smoke-test слоёв без Django
│   ├── container.py     # DI-контейнер (dependency-injector)
│   └── warehouse/
│       └── common/      # constants, exceptions, logger, security, dto/, mappers/,
schemas/
├── dal/                 # database.py, unit_of_work.py, entities/, repositories/
├── bll/                 # interfaces/ (ABC), services/ (реализации)
├── test/                # unittest/pytest-тесты BLL
│   ├── conftest.py      # пусто
│   └── bll/services/employee_service/test_employee_service.py
```

2.2 Корневые файлы

2.2.1 `manage.py`

Что это. Стандартная точка входа команд Django (`runserver`, `test`, `migrate`), дополненная двумя шагами.

Какую задачу решает.

1. Если `.env` нет, а `.env.example` есть, копирует шаблон в `.env` и печатает `[INFO]`. Новый разработчик может сразу выполнить `python manage.py runserver`.
2. Добавляет `src/` в начало `sys.path`, чтобы пакеты `warehouse` и `container` импортировались как модули верхнего уровня.

```
env_path = os.path.join(current_dir, '.env')
example_path = os.path.join(current_dir, '.env.example')
if not os.path.exists(env_path) and os.path.exists(example_path):
    shutil.copy(example_path, env_path)           # первый запуск: .env из шаблона

os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'config.settings')
src_path = os.path.join(current_dir, 'src')
if src_path not in sys.path:
    sys.path.insert(0, src_path)                  # import warehouse.* и container
```

Почему так. Каталог `src/` исторически был самостоятельным проектом (см. `src/main.py`), и все его импорты абсолютные (`from warehouse.dal ...`). Добавить `src` в путь проще, чем переписывать импорты на `src.warehouse...`.

Связи. `config/settings.py` делает то же самое со `SRC_DIR`. Поэтому `wsgi.py` и `asgi.py` работают, даже если их запускают не через `manage.py`.

Альтернативы. Установить `src` как пакет (`pip install -e .` с `pyproject.toml`) — это стандартный подход *src-layout*, он избавляет от правки `sys.path`. Автокопирование `.env` удобно в разработке. На сервере оно опасно: `.env` с пустым `SECRET_KEY` появится молча. Спасает только то, что при `DEBUG=0` `env_secret` упадёт с `ImproperlyConfigured`.

2.2.2 `config/settings.py`

Разобран подробно в главе 14. Основные решения:

- `load_dotenv(BASE_DIR / '.env')` — секреты берутся из окружения;
- `env_secret()` — при `DEBUG=1` отсутствующий секрет заменяется случайным, при `DEBUG=0` приложение падает;
- `INSTALLED_APPS` **не содержит** `django.contrib.auth` и `django.contrib.admin`: пользователи живут в PostgreSQL, а не в Django;
- `DATABASES` — только SQLite для сессий;
- `SESSION_COOKIE_AGE = JWT_EXPIRE_MINUTES * 60` — сессия живёт столько же, сколько токен.

2.2.3 config/urls.py

Одна строка `path('', include('web.urls'))`: все маршруты принадлежат приложению `web`. Админки нет, потому что нет `django.contrib.admin`.

2.2.4 config/wsgi.py И config/asgi.py

Сгенерированы `startproject` и не менялись. В docstring осталось «for server project»: проект раньше назывался `server` (КОММИТ `rename server-config, move to web files`). Для production используется WSGI (`gunicorn/uWSGI`). ASGI бесполезен, пока сессия SQLAlchemy привязана к потоку (`scoped_session`), а синхронные view в async-режиме выполняются в пуле потоков.

2.2.5 requirements.txt

Разобран в главе 19. Сразу отметим: версии `pydantic==2.10.6` и `dependency-injector==4.45.0` **не собираются под Python 3.14**, это проверено при подготовке книги. Под 3.14 нужны `dependency-injector>=4.48` и `pydantic>=2.12`.

2.2.6 sql_fullset.txt И erd.script

- `sql_fullset.txt` — источник истины для схемы БД. В одной транзакции создаются справочники, основные таблицы с CHECK-ограничениями и индексами, загружаются начальные данные и выравниваются счётчики IDENTITY через `setval`.
- `erd.script` — более ранний DDL из pgAdmin ERD Tool. В нём есть таблица `operation_history` (`details jsonb`), которой **нет** в `sql_fullset.txt`. Зато в нём нет `StageDocuments` и CHECK-ограничений. Два скрипта разошлись; нужно оставить один (глава 11).

2.2.7 .env.example, .gitignore, LICENSE.md, README.md

- `.env.example` — все переменные с безопасными значениями по умолчанию и комментариями («DEBUG=1 только на своей машине», «ключ подписи токенов входа: один на все процессы сервера»).
- `.gitignore` — стандартный шаблон GitHub для Python, дополненный `.DS_Store`, `media/` и `staticfiles/`.
- `README.md` — пустой заголовок. Эта книга может стать основой README.

2.3 Организация приложений и их взаимодействие

В Django-части одно приложение — `web`. Оно регистрируется как `web.apps.WebConfig`, чтобы выполнялся `ready()`:

```
class WebConfig(AppConfig):
    default_auto_field = 'django.db.models.BigAutoField'
    name = 'web'

    def ready(self) -> None:
        """Включает логирование проекта при старте Django."""
        from warehouse.common.logger import setup_logging # импорт внутри: после
```

Предметная область лежит вне Django, в пакете `warehouse` (`src/warehouse`). Взаимодействие строго однонаправленное:



`web` импортирует из `warehouse.common` только DTO, исключения и константы, а из `warehouse.bll.interfaces` — только DTO токена. Репозитории и модели `view` не трогают. Исключение — `web/middleware.py`: он импортирует `scoped_session_factory` из DAL, чтобы закрыть сессию в конце запроса.

2.4 Структура шаблонов и статических файлов

```
web/templates/
├── 403.html, 404.html      # страницы ошибок; Django ищет их в корне шаблонов
└── web/
    ├── base.html          # каркас: шапка, блоки, toast-сообщения
    ├── stub.html          # «Раздел в разработке»
    └── auth/
        ├── login.html     # вход (без шапки: {% block appbar %}{% endblock %})
        └── register.html  # регистрация сотрудника
web/static/web/app.css     # 232 строки: токены цвета, светлая и тёмная тема,
компоненты
```

Шаблоны разложены по пространству имён `web/`, по рекомендации Django: это исключает конфликты имён между приложениями. `403.html` и `404.html` лежат в корне, потому что обработчики `handler403/handler404` по умолчанию ищут именно `403.html` и `404.html`.

2.5 Файлы конфигурации и окружения

Файл	Кто читает	Что содержит
<code>.env</code>	<code>config/settings.py</code> , <code>dal/database.py</code>	<code>DB_*</code> , <code>DEBUG</code> , <code>SECRET_KEY</code> , <code>ALLOWED_HOSTS</code> , <code>JWT_SECRET</code> , <code>JWT_EXPIRE_MINUTES</code>
<code>config/settings.py</code>	Django	Всё остальное
<code>src/container.py</code>	<code>web/services.py</code> , <code>src/main.py</code>	Граф зависимостей BLL
<code>sql_fullset.txt</code>	администратор БД (<code>psql -f</code>)	Схема и начальные данные

3 Модели данных

3.1 Где живут модели

В проекте **нет Django-моделей**: `web/models.py` пуст, `web/migrations/` содержит только `__init__.py`. Все сущности — декларативные модели SQLAlchemy 2.0 в `src/warehouse/dal/entities/`. Они наследуются от общего `Base(DeclarativeBase)` и используют типизированный синтаксис `Mapped[...] / mapped_column(...)`.

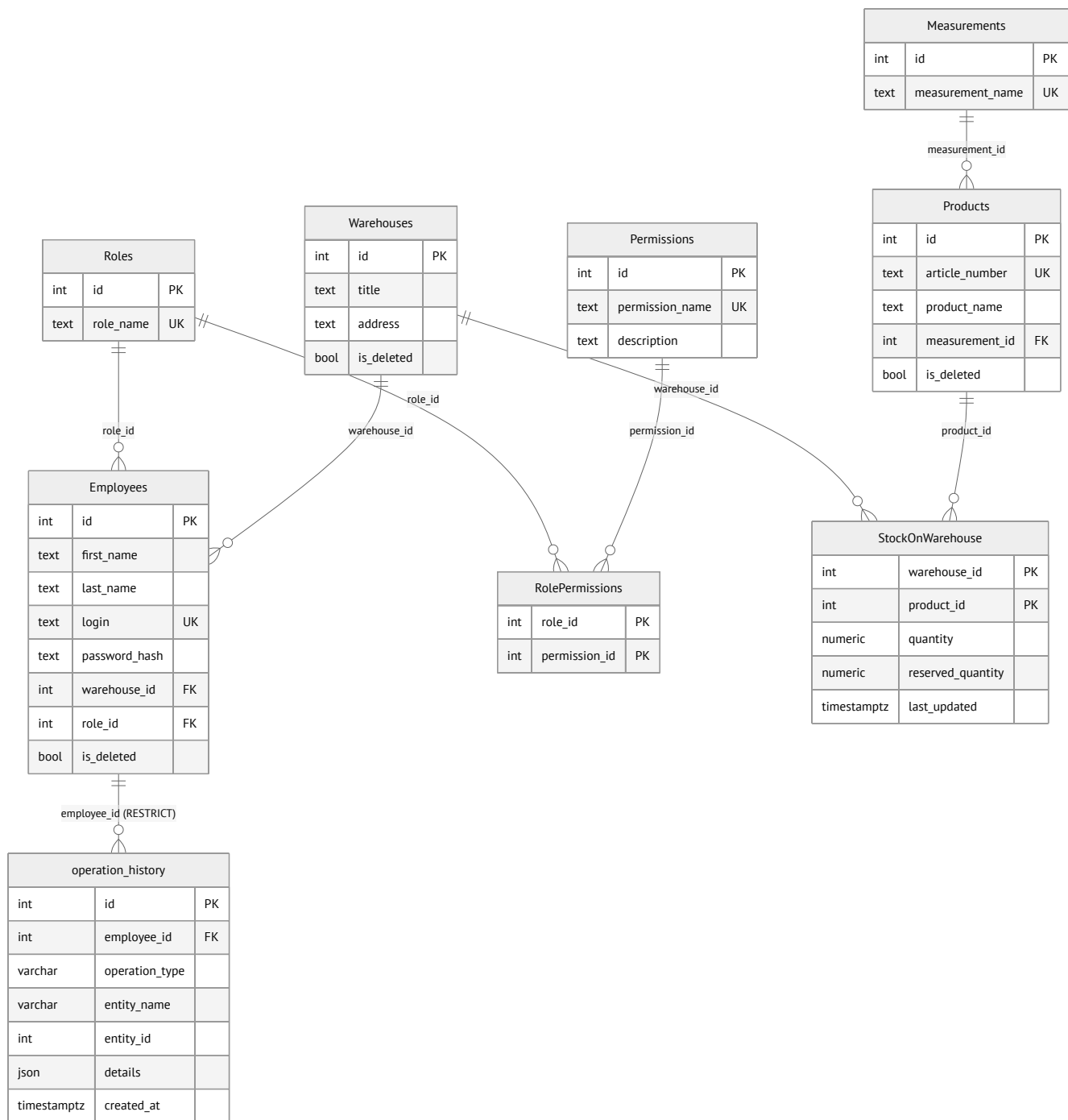
Термин. *Декларативная модель* — класс Python, атрибуты которого описывают столбцы таблицы. SQLAlchemy по ним строит `Table` и маппинг «строка ↔ объект».

Имена таблиц — в стиле PascalCase в кавычках (`"Employees"`, `"ShipmentStages"`), кроме `operation_history`. Схема создавалась в pgAdmin раньше кода, и модели подстраивались под уже существующие имена.

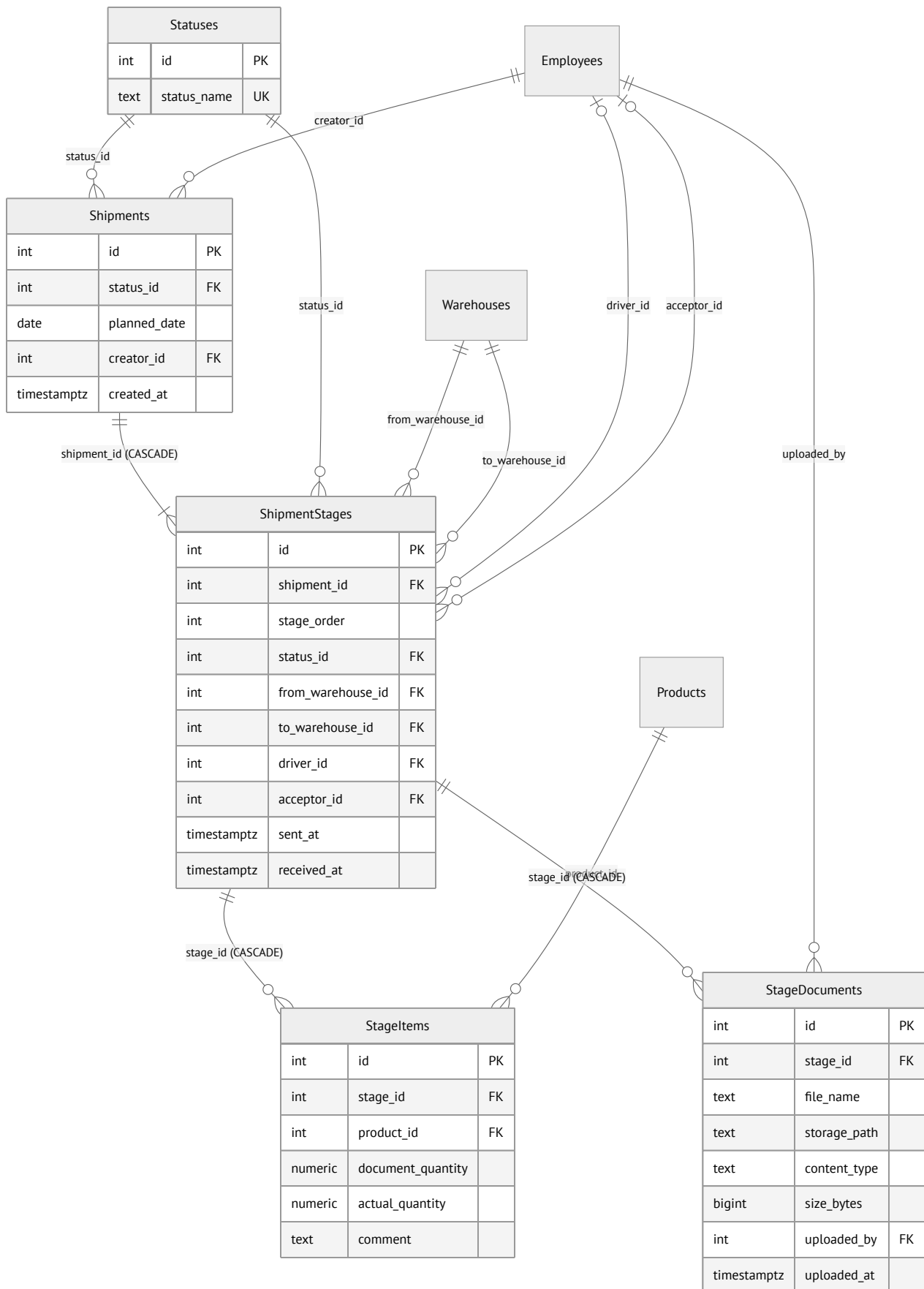
3.2 ER-диаграмма

Схема разбита на две части: справочники и участники, затем перевозки. В части 2 склады, сотрудники и товары показаны без атрибутов.

Часть 1. Справочники, склады, сотрудники, товары и остатки



Часть 2. Перевозки, этапы, позиции и документы



Такая же схема в визуальном виде есть в [documentation/Untitled.png](#) (dbdiagram.io).

3.3 Справочники (entities/reference.py)

3.3.1 Role, Permission, RolePermission

```
class RolePermission(Base):
    __tablename__ = "RolePermissions"
    role_id: Mapped[int] = mapped_column(ForeignKey("Roles.id", ondelete="CASCADE"),
primary_key=True)
    permission_id: Mapped[int] = mapped_column(
        ForeignKey("Permissions.id", ondelete="CASCADE"), primary_key=True, index=True)

class Role(Base):
    __tablename__ = "Roles"
    id: Mapped[int] = mapped_column(Identity(), primary_key=True)
    role_name: Mapped[str] = mapped_column(Text, unique=True)
    permissions: Mapped[list["Permission"]] = relationship(secondary="RolePermissions",
back_populates="roles")
    employees: Mapped[list["Employee"]] = relationship(back_populates="role")
```

- **Связь.** Many-to-Many «роль ↔ право» через явную таблицу-связку RolePermissions с составным первичным ключом. Индекс по permission_id нужен для обратного поиска «у каких ролей есть право». Прямой поиск покрывает первичный ключ (role_id, permission_id).
- **Почему Identity(), а не serial.** GENERATED BY DEFAULT AS IDENTITY — стандарт SQL, рекомендуемый PostgreSQL вместо serial. BY DEFAULT позволяет seed-скрипту вставлять явные id.
- **Почему Text, а не String(n).** В PostgreSQL text и varchar(n) работают одинаково, а искусственный лимит длины ничего не даёт. Непустоту проверяет CHECK (btrim(x) <> '') в DDL.

Роли и права из начальных данных:

Роль (id)	create	accept	dispatch	cancel	view_all	employee:manage
Старший кладовщик (1)	✓	✓	✓			
Администратор системы (3)	✓	✓	✓	✓	✓	✓
Кладовщик (4)		✓	✓			
Менеджер (5)				✓	✓	
Водитель (6)						

Id 2 намеренно пропущен: в уже развёрнутых базах там может остаться удалённая роль «Оператор приёмки» (коммит Убран оператор приемки (добавлен по ошибке)).

3.3.2 Status и Measurement

Простые справочники id + уникальное имя. Статусы общие для перевозки и этапа:

id	status_name	Константа
1	Черновик (Draft)	StatusName.DRAFT
2	Отправлено (Shipped)	SHIPPED

id	status_name	Константа
3	Принято (Received)	RECEIVED
4	Зарезервировано (Reserved)	RESERVED
5	В ожидании (In Waiting)	WAITING
6	Принято с расхождениями (Discrepancy)	DISCREPANCY
7	На транзитном складе (In Transit WH)	IN_TRANSIT_WH
8	Отменено (Cancelled)	CANCELLED

Неочевидное решение. Код никогда не использует числовые id статусов, только имена (`uow.statuses.get_id(StatusName.DRAFT)`). Docstring `StatusRepository` поясняет: «id в БД не 1, 2, 3...». В разных развёртываниях id могли разойтись. Цена решения — лишний `SELECT` на каждый `get_id`. Оптимизация описана в главе 16.

3.4 Warehouse (entities/warehouse.py)

Поле	Тип	Назначение
id	int, Identity, PK	
title	text	Название («Транзитный Сортировочный Узел-1»)
address	text	Адрес
is_deleted	bool, server_default=false()	Мягкое удаление (soft delete)

Связи: `employees` (1:N) и `stock` (1:N). Мягкое удаление выбрано потому, что на склад ссылаются исторические перевозки: физическое удаление сломало бы внешние ключи и историю.

3.5 Employee (entities/employee.py)

```
class Employee(Base):
    __tablename__ = "Employees"
    id: Mapped[int] = mapped_column(Identity(), primary_key=True)
    first_name: Mapped[str] = mapped_column(Text)
    last_name: Mapped[str] = mapped_column(Text)
    login: Mapped[str] = mapped_column(Text, unique=True) # уникален и среди
    удалённых
    password_hash: Mapped[str] = mapped_column(Text) # bcrypt, 60 символов
    warehouse_id: Mapped[int] = mapped_column(ForeignKey("Warehouses.id"), index=True)
    role_id: Mapped[int] = mapped_column(ForeignKey("Roles.id"), index=True)
    is_deleted: Mapped[bool] = mapped_column(default=False, server_default=false())

    warehouse: Mapped["Warehouse"] = relationship(back_populates="employees")
    role: Mapped["Role"] = relationship(back_populates="employees")

    @property
    def full_name(self) -> str: # «Фамилия Имя» — формат для интерфейса
        return f"{self.last_name} {self.first_name}"
```

- У сотрудника **ровно одна роль и ровно один склад**. Это упрощение: сотрудник, работающий на двух складах, потребует таблицу-связку.

- `is_deleted` — «заблокирован». `EmployeeRepository._select()` по умолчанию фильтрует удалённых. Поэтому заблокированный сотрудник не войдёт в систему, а `AccessService.get_actor` выдаст `NotFoundError`.
- `default=False` и `server_default=False()` одновременно: первое для объектов Python до `flush`, второе для вставок в обход ORM.

3.6 Product И StockOnWarehouse

`Product`: `article_number` (артикул, уникальный), `product_name`, `measurement_id` → `Measurements`, `is_deleted`.

`StockOnWarehouse` — остаток товара на складе:

```
class StockOnWarehouse(Base):
    __tablename__ = "StockOnWarehouse"
    __table_args__ = (
        CheckConstraint("reserved_quantity <= quantity",
                        name="ck_StockOnWarehouse_reserved_le_quantity"), # резерв не
    больше остатка
    )
    warehouse_id: Mapped[int] = mapped_column(ForeignKey("Warehouses.id"),
primary_key=True)
    product_id: Mapped[int] = mapped_column(ForeignKey("Products.id"), primary_key=True,
index=True)
    quantity: Mapped[Decimal] = mapped_column(Numeric(12, 3), default=0,
server_default="0")
    reserved_quantity: Mapped[Decimal] = mapped_column(Numeric(12, 3), default=0,
server_default="0")
    last_updated: Mapped[datetime] = mapped_column(
        DateTime(timezone=True), server_default=func.now(), onupdate=func.now())
```

- **Составной РК** (`warehouse_id`, `product_id`). На складе одна строка на товар. Это естественный ключ, и суррогатный `id` не нужен.
- **`Numeric(12, 3)` и `Decimal`**. Товар бывает весовым (кг, т) и объёмным (л). Три знака дают точность до грамма и миллилитра. `float` здесь недопустим из-за ошибок округления: `0.1 + 0.2 != 0.3`.
- **Двойная защита**. `CHECK reserved_quantity <= quantity` в БД — последняя линия обороны. BLL проверяет то же самое заранее, чтобы показать понятное сообщение вместо `IntegrityError` при коммите.

3.7 Перевозки (entities/shipment.py)

3.7.1 Shipment — «шапка» перевозки

Поле	Тип	Смысл
<code>status_id</code>	FK → <code>Statuses</code>	Общий статус перевозки
<code>planned_date</code>	<code>date</code>	Плановая дата отправки

Поле	Тип	Смысл
<code>creator_id</code>	FK → Employees	Кто создал черновик
<code>created_at</code>	<code>timestampz, server_default=now()</code>	Когда
<code>stages</code>	<code>relationship(order_by=stage_order, cascade="all, delete-orphan", passive_deletes=True)</code>	Этапы по порядку

`passive_deletes=True` сообщает SQLAlchemy, что удаление дочерних строк выполнит сама БД (ON DELETE CASCADE). ORM не будет загружать этапы, чтобы удалить их по одному.

3.7.2 ShipmentStage — этап (путь A → B)

Ключевые поля: `shipment_id`, `stage_order` (1..N), `status_id`, `from_warehouse_id`, `to_warehouse_id`, `driver_id` (кто везёт), `acceptor_id` (кто принял), `sent_at`, `received_at`.

- `UniqueConstraint(shipment_id, stage_order)` — в перевозке не бывает двух «этапов №2».
- К таблице `Warehouses` ведут два внешних ключа, к `Employees` — тоже два. Поэтому каждому `relationship` явно указан `foreign_keys=[...]`, иначе SQLAlchemy не поймёт, по какому FK строить связь.
- В DDL есть ещё два ЧЕК, которых нет в модели: `from_warehouse_id <> to_warehouse_id` и `received_at >= sent_at`. Модель не является источником схемы (миграций нет), поэтому расхождение не мешает. Но `Base.metadata.create_all()` создаст таблицу без этих проверок.

3.7.3 StageItem — позиция этапа

`document_quantity` — сколько указано в документах, `actual_quantity` — сколько приняли (NULL, пока факт не внесён), `comment` — объяснение расхождения. `UniqueConstraint(stage_id, product_id)`: один товар — одна позиция в этапе. Поэтому повторный `add_item` того же товара **заменяет** количество, а не добавляет вторую строку.

3.7.4 StageDocument — документ этапа

Хранит только метаданные файла: `file_name` (исходное имя), `storage_path` (путь в `default_storage`), `content_type`, `size_bytes`, `uploaded_by`, `uploaded_at`. Сам файл хранит web-слой. Docstring предупреждает: при каскадном удалении этапа строка исчезнет, **а файл на диске останется**. Это осознанный компромисс: DAL не должен знать о файловой системе.

3.8 OperationHistory (entities/operation_history.py)

Журнал аудита: кто (`employee_id`, ON DELETE RESTRICT), что (`operation_type`, `entity_name`, `entity_id`), подробности (`details`: JSON), когда.

Состояние. Модель и репозиторий `HistoryRepository.log_operation()` есть, и `uow.history` создаётся. Но ни один сервис его не вызывает, а в `sql_fullset.txt` нет `CREATE TABLE operation_history`. Комментарий в `ShipmentReceiptService._stage_on_my_warehouse` объясняет, почему отказы в доступе пишутся в лог, а не в `uow.history`: запись откатилась бы вместе с транзакцией. Остальной стиль модели тоже выбивается: `Integer` вместо `Identity()`, `String(50)`, `JSON` вместо `JSONB`, `index=True` на первичном ключе (избыточно, РК уже индексирован).

3.9 Методы моделей, менеджеры и QuerySet

В Django логику выборки кладут в менеджеры (`objects`) и кастомные `QuerySet`. Здесь эту роль играют **репозитории** (глава 11): у каждого есть приватный `_select()`. Это аналог «базового `QuerySet`» с нужными `joinedload` и фильтром `is_deleted`:

```
class ProductRepository(BaseRepository[Product]):
    model = Product

    def _select(self):
        Product.objects.active().select_related('measurement')
        return (select(Product)
                .options(joinedload(Product.measurement, innerjoin=True))
                .where(Product.is_deleted.is_(False)))
```

Единственные методы на самих моделях — вычисляемое свойство `Employee.full_name`. Аналогичные свойства есть у DTO: `EmployeeDTO.full_name`, `StockItemDTO.available`.

3.10 DTO — «модели» для остальных слоёв

`src/warehouse/common/dto/` — замороженные `dataclass(frozen=True, slots=True)`:

DTO	Назначение
<code>EmployeeDTO</code> , <code>EmployeeAuthDTO</code> , <code>NewEmployee</code> , <code>RoleDTO</code>	Сотрудник, сотрудник с хэшем (только для входа), форма регистрации, роль
<code>WarehouseDTO</code> , <code>StockItemDTO</code> (+ <code>available</code>)	Склад, остаток
<code>ProductDTO</code>	Товар с названием единицы измерения
<code>ShipmentDTO</code> , <code>StageDTO</code> , <code>StageItemDTO</code> , <code>StageDocumentDTO</code>	Результаты чтения перевозки
<code>NewStage</code> , <code>NewStageItem</code> , <code>NewStageDocument</code>	Входные данные для записи

`frozen=True` делает объект неизменяемым: `view` не сможет «поправить» статус в DTO и ошибочно решить, что он сохранился. `slots=True` экономит память и запрещает опечатки в именах атрибутов.

Перевод «модель → DTO» выполняют функции из `common/mappers/`: `to_employee`, `to_stage`, `to_shipment` и другие. Их **можно вызывать только при открытой сессии**, потому что они обходят связи (`st.shipment.status.status_name`).

3.11 Миграции: история и структура

Миграций как механизма **нет**. Ни Django-миграций (нет моделей), ни Alembic. Схема эволюционировала так: pgAdmin ERD → `erd.script` → ручной `sql_fullset.txt` с `CREATE TABLE IF NOT EXISTS` и `ON CONFLICT DO NOTHING`, чтобы скрипт можно было перезапускать.

Последствия:

- `IF NOT EXISTS` **не изменит** существующую таблицу. Новый столбец или `CHECK` придётся добавлять вручную через `ALTER TABLE`;
- схема в моделях и схема в SQL могут разойтись незаметно, и уже расходятся: `operation_history`, `CHECK` у этапов.

Рекомендация — **Alembic** с `autogenerate` от `Base.metadata`. Порядок внедрения описан в главе 18.

Таблица сессий Django создаётся обычной командой `python manage.py migrate` (миграции `contenttypes`, `sessions`) в SQLite.

4 Представления (Views)

4.1 Обзор и принцип «тонких» view

Все view лежат в пакете `web/views/`. Модули разбиты по экранам будущего интерфейса, и эти экраны один к одному повторяют `documentation/prototype.html`:

Модуль	Экран	Состояние
<code>auth.py</code>	Вход, выход, регистрация сотрудника	Реализован
<code>api.py</code>	JSON API	Реализован
<code>home.py</code>	Главная, остатки	Заглушка
<code>shipments.py</code>	Список перевозок	Заглушка
<code>draft.py</code>	Новая перевозка и правка черновика	Заглушка
<code>detail.py</code>	Карточка перевозки: резерв, отправка, отмена, документ	Заглушка
<code>receipt.py</code>	Приёмка	Заглушка
<code>_stub.py</code>	Общие заглушки	Служебный

Главный принцип записан в docstring `web/forms.py`: «**только разбор полей, бизнес-проверки делают сервисы**». View:

1. проверяет вход (`@employee_required`);
2. берёт `employee_id` из `request.actor`;
3. вызывает **один** метод сервиса;
4. превращает результат или `BusinessError` в HTML/JSON.

Права, статусы, остатки и принадлежность складу view не проверяет никогда. Это делает BLL, поэтому HTML-страница и JSON API не могут разойтись в правилах.

4.2 Class-based или function-based views

- **HTML-страницы — функции (FBV)**. Каждая страница — 5–15 строк без наследования. Декораторы `@require_GET`, `@employee_required`, `@business_errors_as_http` видны сразу над функцией. `ListView/DetailView` здесь бесполезны: они завязаны на Django ORM (`queryset`, `get_object`), а данные приходят из сервисов в виде DTO.
- **API — классы (CBV) на базе `django.views.View`**. Класс удобно разделяет методы HTTP. Внедрение сервисов через `@inject + Provide[...]` естественно ложится на аргументы метода. Декоратор навешивается на `dispatch` через `method_decorator`.

4.3 Страницы аутентификации (web/views/auth.py)

4.3.1 login_view

```
@require_http_methods(['GET', 'POST'])
def login_view(request):
    """Страница входа по логину и паролю."""
    if request.method == 'GET' and authenticate(request) is not None:
        return redirect('home') # уже вошёл — на главную

    next_url = request.POST.get('next') or request.GET.get(REDIRECT_FIELD_NAME, '')
    context = {'login': '', 'error': None, 'next': next_url}

    if request.method == 'POST':
        context['login'] = request.POST.get('login', '').strip() # логин вернём в форму
        # при ошибке
        try:
            token = login_service().login(context['login'], request.POST.get('password', ''))
            warehouse = employee_service().get_warehouse(token.employee.id) # название склада для шапки
        except BusinessError as e:
            context['error'] = e.message # «Неверный логин или пароль»
        else:
            sign_in(request, token, warehouse)
            return redirect(_safe_next(request, next_url)) # защита от open redirect
    return render(request, 'web/auth/login.html', context)
```

Здесь две важные детали.

- `_safe_next` пропускает `next` только через `url_has_allowed_host_and_scheme`. Иначе ссылка `../login/?next=https://evil.example/` отправила бы сотрудника на фишинговый сайт сразу после входа. Этот случай покрывает тест `test_next_to_other_host_is_ignored`.
- Пароль при ошибке в форму **не возвращается**, логин — возвращается. Это видно в `test_wrong_password_keeps_login`.

4.3.2 logout_view

Принимает только `@require_POST`. Выход по GET-ссылке позволил бы стороннему сайту разлогинить пользователя картинкой ``. Кнопка выхода в `base.html` — это форма с `{% csrf_token %}`.

4.3.3 register_view

```
@require_http_methods(['GET', 'POST'])
@employee_required
@business_errors_as_http # AccessDeniedError из list_warehouses → 403
def register_view(request):
    admin_id = request.actor.employee_id
    service = employee_service()
    options = {'warehouses': service.list_warehouses(admin_id), 'roles':
service.list_roles(admin_id)}
```

```

error = None
if request.method == 'POST':
    form = RegisterForm(request.POST, **options)
    if form.is_valid():
        try:
            employee = service.register(admin_id, form.to_new_employee())
        except BusinessException as e:
            error = e.message # «Логин ... уже занят» –
над формой
        else:
            messages.success(request, f'Сотрудник {employee.full_name}
зарегистрирован, ...')
            return redirect('employee_new') # PRG: Post/Redirect/Get
    else:
        form = RegisterForm(**options)
    return render(request, 'web/auth/register.html', {'form': form, 'error': error})

```

Права администратора view не проверяет сам. Их проверяет `list_warehouses`: без `employee:manage` он бросит `AccessDeniedError`, а декоратор превратит её в 403. После успешной регистрации используется шаблон **PRG** (*Post/Redirect/Get*): обновление страницы не создаст сотрудника повторно.

4.4 Заглушки (`_stub.py`) и будущие страницы

```

def stub_page(request, title: str):
    return render(request, 'web/stub.html', {'title': title}) # «Раздел в
разработке»

def stub_action(request, title: str):
    messages.info(request, f'{title}: в разработке')
    return redirect_back(request) # назад, откуда нажали
кнопку

```

Все будущие view уже объявлены с правильными методами HTTP, декораторами и сигнатурами (`stage_id`, `shipment_id`...). Тест `test_every_stub_page_opens` проверяет, что каждая страница открывается. Реализация сведётся к замене тела функции. Пример того, как должен выглядеть `reserve_view` (кнопка, POST):

```

@require_POST
@employee_required
@business_errors_as_messages # InvalidStatusError → messages.error + редирект
назад
def reserve_view(request, stage_id: int):
    dispatch_service().reserve_stage(request.actor.employee_id, stage_id)
    messages.success(request, 'Этап зарезервирован')
    return redirect_back(request)

```

4.5 JSON API (`web/views/api.py`)

Класс	Метод	Сервис
LoginView	POST	login_service.login + employee_service.get_warehouse → sign_in

Класс	Метод	Сервис
StockListView	GET	<code>draft_service.list_available_stock</code>
CreateDraftView	POST	<code>draft_service.create_draft</code>
ShipStageView	POST	<code>dispatch_service.ship_stage</code>
AcceptStageView	POST	<code>receive_service.accept_stage</code>
CancelShipmentView	POST	<code>dispatch_service.cancel_shipment</code>

Общие помощники:

```
def _json_body(request) -> dict:
    try:
        body = json.loads(request.body)
    except json.JSONDecodeError:
        raise ValidationError("Невалидный JSON") from None    # → middleware → 400 JSON
    if not isinstance(body, dict):
        raise ValidationError("Ожидается JSON-объект")
    return body

def _json(data, status: int = 200) -> JsonResponse:
    # dataclasses.asdict рекурсивно раскрывает вложенные DTO; DjangoJSONEncoder умеет
    # Decimal/date/datetime
    return JsonResponse(asdict(data), encoder=DjangoJSONEncoder, status=status)
```

Внедрение зависимостей — через *wiring dependency-injector*:

```
@method_decorator(api_employee_required, name='dispatch')    # без входа → 401 JSON
class ShipStageView(View):
    @inject
    def post(self, request, stage_id: int, *args,
             dispatch_service: ShipmentDispatchService =
Provide[Container.dispatch_service], **kwargs):
        return _json(dispatch_service.ship_stage(employee_id=request.actor.employee_id,
stage_id=stage_id))

container.wire(modules=[__name__])    # в конце модуля: подменяет Provide[...] реальными
провайдерами
```

Замечания к API.

1. В проекте два стиля получения сервиса: `services.draft_service()` в HTML-view и `Provide[Container.draft_service]` в API. Работают оба, и оба подменяются в тестах через `container.<provider>.override(...)`. Но лучше выбрать один: проще — функции из `web/services.py`.
2. `LoginView` помечен `@csrf_exempt`, а остальные POST — нет. Значит, JSON-клиент должен сначала получить cookie `csrftoken` и передавать заголовок `X-CSRFToken`. Это правильно для сессионной аутентификации. `csrf_exempt` на входе открывает возможность *login CSRF* (подробнее в главе 15).
3. В `CreateDraftView` значение `route` не приводится к `int`. Для строки `"1"` вызов `require_warehouse(actor, "1")` не совпадёт с `warehouse_id == 1`, и не-администратор получит 403 вместо понятной ошибки валидации.

4.6 Декораторы, применяемые к view

Декоратор	Файл	Что делает
<code>require_GET / require_POST / require_http_methods</code>	Django	Иначе 405
<code>employee_required</code>	<code>web/auth.py</code>	Нет валидного токена → редирект на <code>/login/?next=...</code> ; для не-GET — без <code>next</code>
<code>api_employee_required</code>	<code>web/auth.py</code>	Нет токена → 401 <code>{"error": "Войдите в систему"}</code>
<code>business_errors_as_http</code>	<code>web/errors.py</code>	<code>NotFoundError</code> → <code>Http404</code> , <code>AccessDeniedError</code> → <code>PermissionDenied (403)</code>
<code>business_errors_as_messages</code>	<code>web/errors.py</code>	Любая <code>BusinessError</code> → <code>messages.error</code> + <code>redirect_back</code>
<code>csrf_exempt</code> , <code>method_decorator</code>	Django	См. выше

Порядок декораторов важен. В `register_view` `@employee_required` стоит над `@business_errors_as_http`: сначала проверяется вход, и только потом ошибки BLL превращаются в HTTP-ответ. Если поменять их местами, ошибки авторизации тоже будут перехвачены, но ради них придётся вызвать сервис до проверки входа.

4.7 Пагинация, фильтрация, сортировка

Пока **не реализованы**. Сортировку сейчас задают репозитории: этапы — `shipment_id DESC`, `stage_order`, сотрудники — `last_name`, товары — `product_name`. Фильтрация по статусам сделана параметром `only_active / status_ids`. В прототипе список перевозок фильтруется чипами и «поиском с токенами» (CSS `.search-box`, `.token` уже в `app.css`). Для пагинации рекомендуется добавить в репозиторий `limit/offset` (или `keyset` по `id`) и возвращать `Page[DTO]`. `django.core.paginator.Paginator` поверх уже загруженного списка выгрузит все строки в память.

5 URL-маршрутизация

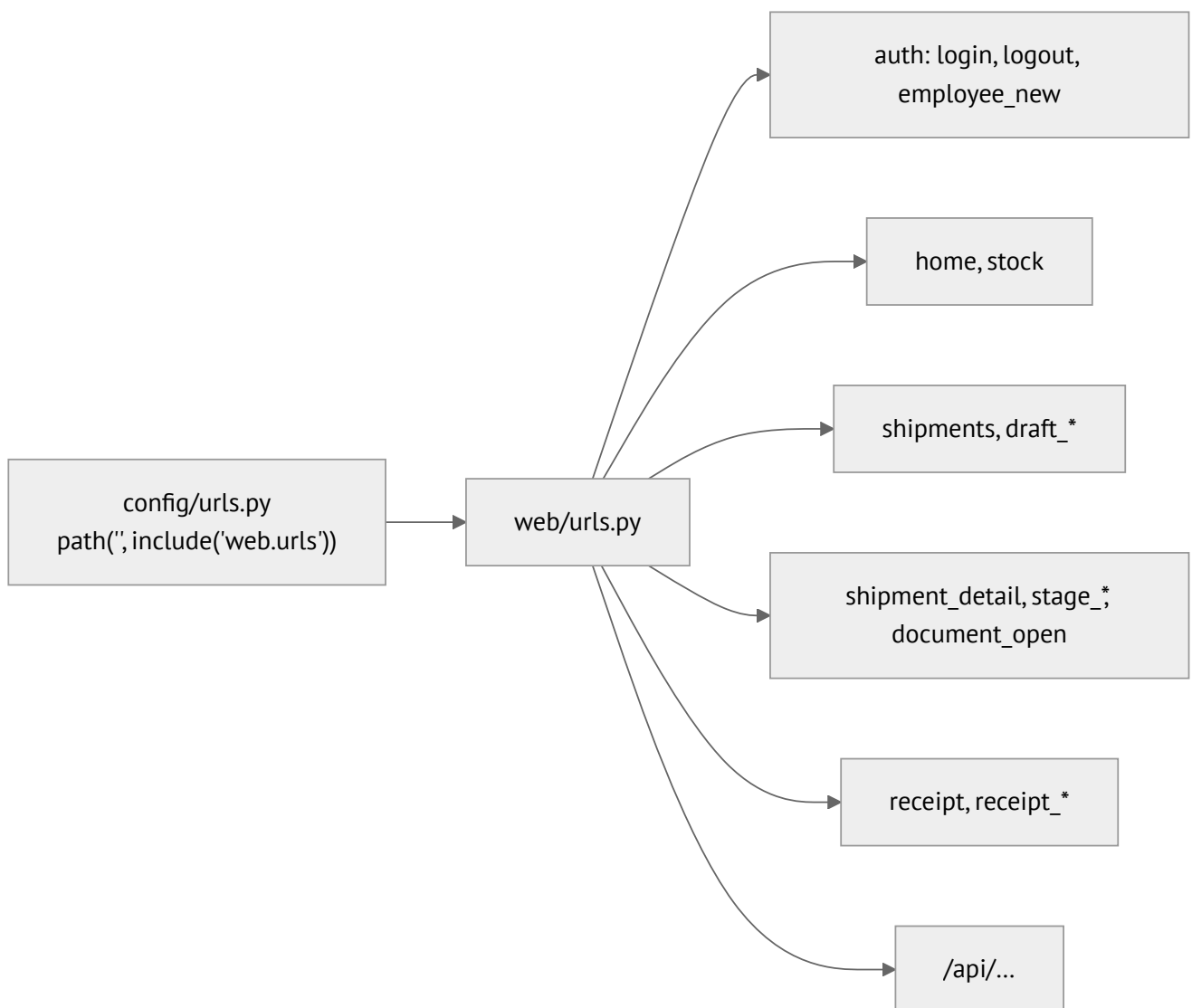
5.1 Полная карта URL

Используется `path()` с конвертером `<int:...>`. Регулярные выражения (`re_path`) не нужны: конвертер `int` сам отбрасывает нечисловые id и отвечает 404.

URL	Имя	View	Метод	Статус
/login/	login	auth.login_view	GET, POST	✓
/logout/	logout	auth.logout_view	POST	✓
/employees/new/	employee_new	auth.register_view	GET, POST	✓
/	home	home.home_view	GET	заглушка
/stock/	stock	home.stock_view	GET	заглушка
/shipments/	shipments	shipments.shipment_list_view	GET	заглушка
/shipments/new/	draft_new	draft.draft_create_view	GET, POST	заглушка
/stages/<int:stage_id>/items/add/	draft_add_item	draft.add_item_view	POST	заглушка
/items/<int:item_id>/remove/	draft_remove_item	draft.remove_item_view	POST	заглушка
/stages/<int:stage_id>/driver/	draft_driver	draft.assign_driver_view	POST	заглушка
/stages/<int:stage_id>/documents/add/	draft_attach	draft.attach_document_view	POST	заглушка
/documents/<int:document_id>/remove/	draft_detach	draft.remove_document_view	POST	заглушка
/shipments/<int:shipment_id>/delete/	draft_delete	draft.delete_draft_view	POST	заглушка
/shipments/<int:shipment_id>/	shipment_detail	detail.shipment_detail_view	GET	заглушка
/stages/<int:stage_id>/reserve/	stage_reserve	detail.reserve_view	POST	заглушка
/stages/<int:stage_id>/ship/	stage_ship	detail.ship_view	POST	заглушка
/shipments/<int:shipment_id>/cancel/	shipment_cancel	detail.cancel_view	POST	заглушка
/stages/<int:stage_id>/documents/<int:document_id>/	document_open	detail.document_open_view	GET	заглушка
/receipt/	receipt	receipt.receipt_list_view	GET	заглушка

URL	Имя	View	Метод	Статус
/stages/<int:stage_id>/facts/	receipt_facts	receipt.save_facts_view	POST	заглушка
/stages/<int:stage_id>/accept/	receipt_accept	receipt.accept_view	POST	заглушка
/api/auth/login/	api_login	api.LoginView	POST	✓
/api/stock/	api_stock	api.StockListView	GET	✓
/api/shipments/draft/	api_draft_create	api.CreateDraftView	POST	✓
/api/stages/<int:stage_id>/ship/	api_stage_ship	api.ShipStageView	POST	✓
/api/stages/<int:stage_id>/accept/	api_stage_accept	api.AcceptStageView	POST	✓
/api/shipments/<int:shipment_id>/cancel/	api_shipment_cancel	api.CancelShipmentView	POST	✓

5.2 Вложенность и принципы именования



- URL строятся от **ресурса**, над которым выполняется действие: `/stages/<id>/reserve/`, `/items/<id>/remove/`. Идентификатор в URL совпадает с аргументом метода сервиса (`reserve_stage(employee_id, stage_id)`).
- Все изменения выполняются через POST на «глагольный» URL. HTML-формы не умеют PUT/DELETE, а разделение на GET (чтение) и POST (изменение) — главное требование к CSRF-защите.
- `app_name` не задан, поэтому имена глобальные (`'login'`, а не `'web:login'`). Для проекта с одним приложением это допустимо. При появлении второго приложения стоит ввести пространство имён.
- Префикс `/api/` важен для middleware: только для таких путей `BusinessError` превращается в JSON (глава 8).

5.3 Именованные URL в шаблонах и коде

```
<a class="brand" href="{% url 'home' %}">...</a>
<form method="post" action="{% url 'logout' %}">{% csrf_token %}...</form>
```

```
return redirect('employee_new') # имя, не путь
redirect(f'{reverse('login')}{urlencode({'next': request.get_full_path()})}')
```

`LOGIN_URL = 'login'` в settings тоже задан именем.

5.4 API endpoints

DRF (*Django REST Framework*) не используется: API — шесть ручных View, которые возвращают `JsonResponse`. Формат ошибок единый — `{"error": "<текст>"}` с HTTP-кодом из `BusinessError.status_code`.

Исключение	HTTP
<code>ValidationError</code>	400
<code>AuthError</code>	401
<code>AccessDeniedError</code>	403
<code>NotFoundError</code>	404
<code>InvalidStatusError</code>	409 Conflict
иная <code>BusinessError</code>	500

Пример сеанса:

```
# 1. вход: токен ляжет в серверную сессию, клиент получит cookie sessionid
curl -c jar -H 'Content-Type: application/json' \
  -d '{"login":"ivan_sklad","password":"..."}' http://localhost:8000/api/auth/login/

# 2. свободные остатки своего склада
curl -b jar http://localhost:8000/api/stock/
# {"stocks": [{"warehouse_id": 1, "product_id": 1, "article_number": "ART-001", ...,
#               "quantity": "5500.750", "reserved_quantity": "0.000"}]}
```

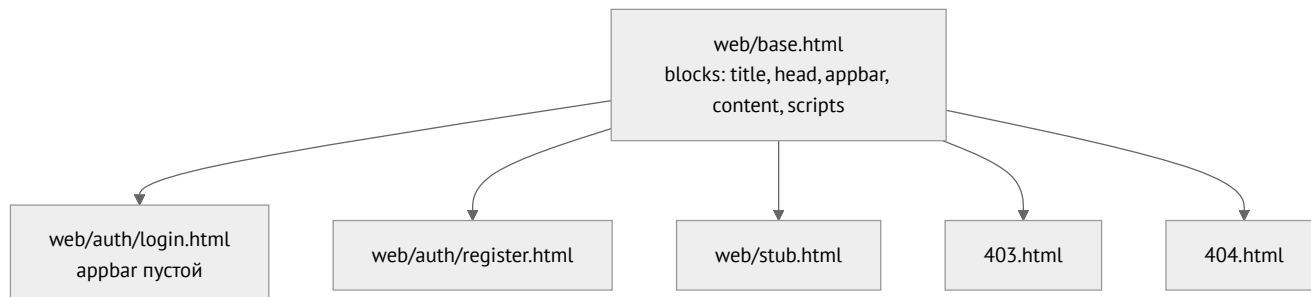
3. POST требует CSRF: взять csrftoken из cookie и передать заголовком

```
curl -b jar -c jar -H "X-CSRFToken: $CSRF" -H 'Content-Type: application/json' \
  -d '{"planned_date":"2026-10-01","route":[1,2,3],"items":
[{"product_id":1,"quantity":"10.5"}]}' \
  http://localhost:8000/api/shipments/draft/
```

Эндпоинт `/api/auth/login/` не отдаёт `csrftoken`, потому что шаблон с `{% csrf_token %}` не рендерится. Чтобы cookie появился, JSON-клиенту сначала нужен GET на `/login/` (или view с `@ensure_csrf_cookie`).

6 Шаблоны (Templates)

6.1 Наследование



6.2 web/base.html

Каркас всех страниц. Он:

- подключает шрифты Onest и IBM Plex Mono из Google Fonts и `{% static 'web/app.css' %}`;
- рисует шапку (`{% block appbar %}`): бренд, ФИО, роль и склад из context processor, кнопку «Новый сотрудник» только при праве `employee:manage` и форму выхода;
- выводит `messages` во всплывающем уведомлении, которое прячется через 4 с (inline-скрипт).

```
{% if 'employee:manage' in perms %}
  <a class="btn btn-round btn-sm" href="{% url 'employee_new' %}">Новый сотрудник</a>
{% endif %}
```

Нюанс с perms. В стандартном Django переменную `perms` в шаблонах даёт `django.contrib.auth.context_processors.auth` (объект `PermWrapper`). Здесь `django.contrib.auth` не установлен, и `perms` определяет собственный процессор `web.context.current_user` как `frozenset` строк. Проверка поэтому пишется как `'employee:manage' in perms`, а не `perms.app.codename`. Кнопка скрывается только ради удобства: реальную проверку выполняет BLL.

6.3 Остальные шаблоны

Шаблон	Назначение	Особенности
web/auth/login.html	Вход	Пустой <code>appbar</code> ; скрытое поле <code>next</code> ; <code>autocomplete="username" / current-password</code> ; ошибка с <code>role="alert"</code>
web/auth/register.html	Регистрация	Поля рендерятся циклом <code>{% for field in form %}</code> с ошибками под каждым полем и <code>non_field_errors</code>
web/stub.html	Заглушка	Заголовок из <code>title</code>

Шаблон	Назначение	Особенности
403.html , 404.html	Ошибки	{{ exception default:'...' }} — текст <code>PermissionDenied</code> / <code>Http404</code> , то есть сообщение из <code>BLL</code>

Сообщение исключения попадает в 403/404 только потому, что `business_errors_as_http` передаёт `e.message` в `Http404(...)/PermissionDenied(...)`, а стандартные обработчики Django кладут `exception` в контекст шаблона. При `DEBUG=1` Django показывает свою отладочную страницу, и кастомный `404.html` не используется.

6.4 Custom template tags и filters (`templatetags/warehouse_tags.py`)

```

STATUS_CLASSES = {StatusName.DRAFT: 'st-draft', StatusName.WAITING: 'st-waiting', ...,
                  StatusName.IN_TRANSIT_WH: 'st-shipped', ...} # транзит красится как «в
пути»
FRACTIONAL_UNITS = {'т', 'кг'}

@register.filter
def status_class(status_name): return STATUS_CLASSES.get(status_name, 'st-waiting')

@register.filter
def status_label(status_name): return (status_name or '').split(' ')[0] # «Черновик
(Draft)» → «Черновик»

@register.filter
def qty(value, unit=''):
    """Количество с единицей: для т и кг три знака после запятой, остальное целым."""
    number = Decimal(str(value))
    digits = 3 if unit in FRACTIONAL_UNITS else 0
    text = number_format(number, decimal_pos=digits, use_l10n=True, force_grouping=True)
    return f'{text} {unit}'.strip()

@register.simple_tag
def status_pill(status_name):
    return format_html('<span class="pill {}">{}</span>', status_class(status_name),
status_label(status_name))

```

Использование и результат (из теста `test_tags`):

```

{% load warehouse_tags %}
{% status_pill stage.status_name %} → <span class="pill st-draft">Черновик</span>
{{ "1234.5"|qty:'т' }}                → 1 234,500 т      (неразрывный пробел, запятая —
локаль ru)
{{ 12|qty:'шт' }}                    → 12 шт

```

`format_html` экранирует аргументы, поэтому имя статуса из БД не может внедрить HTML. Литры ('л') не входят в `FRACTIONAL_UNITS`, хотя в БД количество хранится с тремя знаками. Значит, 1,5 л покажутся как «2 л». Стоит добавить 'л' в набор.

6.5 Context processors

```
def current_user(request) -> dict:
    """employee_name, role_name, warehouse_title и perms для шапки и кнопок."""
    actor = getattr(request, 'actor', None) # ставит employee_required
    if actor is None:
        return {'perms': frozenset()}
    return {'employee_name': request.session.get('employee_name', ''),
            'role_name': actor.role_name,
            'warehouse_title': request.session.get('warehouse_title', ''),
            'perms': actor.permissions}
```

ФИО и название склада кэшируются в сессии при входе (`sign_in`). Так шапка не делает запросов в PostgreSQL на каждой странице. Цена — после переименования склада шапка покажет старое название до следующего входа.

6.6 Стили (`static/web/app.css`) и frontend

Сборщиков и JS-фреймворков нет. Один CSS-файл построен на **дизайн-токенах** — CSS-переменных `--bg`, `--accent`, `--draft-bg`... Тёмная тема включается через `prefers-color-scheme` или атрибутом `data-theme`. Там же адаптивная вёрстка на 760 px и `prefers-reduced-motion`. Классы (`.counter`, `.tile`, `.route`, `.stage`, `.fact-input`, `.drop`) заранее перенесены из прототипа для будущих экранов.

7 Формы (Forms)

7.1 RegisterForm

Единственная форма проекта. Это обычная `forms.Form`, а не `ModelForm`: Django-модели сотрудника не существует.

```
class RegisterForm(forms.Form):
    last_name = forms.CharField(label='Фамилия', max_length=100)
    first_name = forms.CharField(label='Имя', max_length=100)
    login = forms.CharField(label='Логин', max_length=100)
    password = forms.CharField(label='Пароль',
    widget=forms.PasswordInput(render_value=False))
    password_repeat = forms.CharField(label='Повтор пароля',
    widget=forms.PasswordInput(render_value=False))
    warehouse_id = forms.TypedChoiceField(label='Склад', coerce=int)    # '1' → 1
    role_id = forms.TypedChoiceField(label='Роль', coerce=int)

    def __init__(self, *args, warehouses: list[WarehouseDTO], roles: list[RoleDTO],
    **kwargs):
        super().__init__(*args, **kwargs)
        # choices заполняются из сервиса: форма не ходит в БД сама
        self.fields['warehouse_id'].choices = [(w.id, w.title) for w in warehouses]
        self.fields['role_id'].choices = [(r.id, r.name) for r in roles]
        self.fields['login'].widget.attrs['autocomplete'] = 'off'
        for name in ('password', 'password_repeat'):
            self.fields[name].widget.attrs['autocomplete'] = 'new-password'    # менеджер
            # паролей не подставит пароль админа
        for field in self.fields.values():
            field.widget.attrs['class'] = 'input'

    def clean(self):
        data = super().clean()
        if data.get('password') and data.get('password') != data.get('password_repeat'):
            self.add_error('password_repeat', 'Пароли не совпадают')
        return data

    def to_new_employee(self) -> NewEmployee:    # форма → входной DTO сервиса
        ...
```

7.2 Разделение валидации

Уровень	Что проверяет	Где
Форма	Поля заполнены, длина ≤ 100, id — число из списка, пароли совпадают	RegisterForm
Сервис	Имя и фамилия непусты после strip(), логин без пробелов, пароль ≥ 8 символов, склад и роль существуют, логин свободен	EmployeeService.register
БД	UNIQUE(login), CHECK (btrim(login) <> ''), FK	sql_fullset.txt

Форма отвечает только за то, что относится к HTTP (совпадение повтора пароля — чисто интерфейсная проверка). Бизнес-правила живут в сервисе, поэтому JSON API или CLI получают ту же проверку.

Несогласованность. В `common/schemas/auth_dto.py` (наследие FastAPI) минимальная длина пароля — 6, а логин ограничен регуляжкой `^[a-zA-Z0-9_]+$`. Действующие правила — в `EmployeeService`: пароль от 8 символов, логин без пробелов, кириллица разрешена. Схемы не используются и вводят в заблуждение.

7.3 Widgets

`PasswordInput(render_value=False)` не возвращает пароль в HTML при повторном показе формы. Остальная кастомизация — атрибуты `class` и `autocomplete`. Собственных виджетов нет.

7.4 Formsets

Не используются. Кандидат — ввод факта при приёмке: `/stages/<id>/facts/` сохраняет сразу все позиции этапа. Сервис принимает позиции по одной (`enter_actual_quantity(item_id, ...)`), поэтому для этого экрана подойдёт `formset_factory` с полями `item_id`, `actual_quantity`, `comment`. Все вызовы сервиса желательно выполнять в одной транзакции (подробнее в главе 21).

7.5 Загрузка файлов (`web/uploads.py`)

```
UPLOAD_DIR = 'stage_documents'
ALLOWED_EXTENSIONS = {'.pdf', '.jpg', '.jpeg', '.png'}

def save_stage_file(uploaded_file) -> NewStageDocument:
    if uploaded_file is None:
        raise ValidationError('Выберите файл')
    file_name = PurePath(uploaded_file.name).name # отбрасываем путь клиента
    extension = PurePath(file_name).suffix.lower()
    if extension not in ALLOWED_EXTENSIONS:
        raise ValidationError(f'Файл «{file_name}»: можно загрузить только PDF, JPG или PNG')
    if uploaded_file.size > settings.MAX_UPLOAD_SIZE: # 10 МБ
        raise ValidationError(f'Файл «{file_name}» больше {limit_mb} МБ')
    # имя на диске — случайный UUID: нет коллизий, нет path traversal, нельзя угадать
    storage_path = default_storage.save(f'{UPLOAD_DIR}/{uuid.uuid4().hex}{extension}',
    uploaded_file)
    return NewStageDocument(file_name=file_name, storage_path=storage_path,
    content_type=uploaded_file.content_type,
    size_bytes=uploaded_file.size)
```

Задуманный сценарий прикрепления документа, с компенсацией при ошибке BLL:

```
@require_POST
@employee_required
```

```

@business_errors_as_messages
def attach_document_view(request, stage_id: int):
    document = save_stage_file(request.FILES.get('file'))      # 1. файл на диск
    try:
        draft_service().attach_document(request.actor.employee_id, stage_id, document) #
    2. строка в БД
    except BusinessException:
        delete_stage_file(document.storage_path)              # 3. откат файла вручную
        raise
    return redirect_back(request)

```

Файловая система не участвует в транзакции БД, поэтому откат делается вручную. При удалении документа порядок обратный: сначала `remove_document` (транзакция), затем `delete_stage_file`. Обратите внимание: `content_type` приходит от клиента и не проверяется. Для строгой проверки нужно читать сигнатуру файла (magic bytes), например через `python-magic`.

8 Аутентификация и авторизация

8.1 Система пользователей

`django.contrib.auth` **не используется**. Нет `User`, `Group`, `Permission` Django, нет `AUTH_USER_MODEL`. Пользователь — строка `Employees` в PostgreSQL. Группы — `Roles`, права — `Permissions` через `RolePermissions`. Имена прав — строки вида `ресурс:действие` из `PermissionName`.

Почему не кастомная модель пользователя Django: вся предметная область на SQLAlchemy, и держать сотрудника одновременно в двух ORM и двух базах означало бы синхронизацию. Цена этого решения — нет готовой админки, `login_required`, `PasswordResetView` и защиты `AUTH_PASSWORD_VALIDATORS`. Всё это написано вручную в `web/auth.py` и `LoginService`.

8.2 Механизм аутентификации

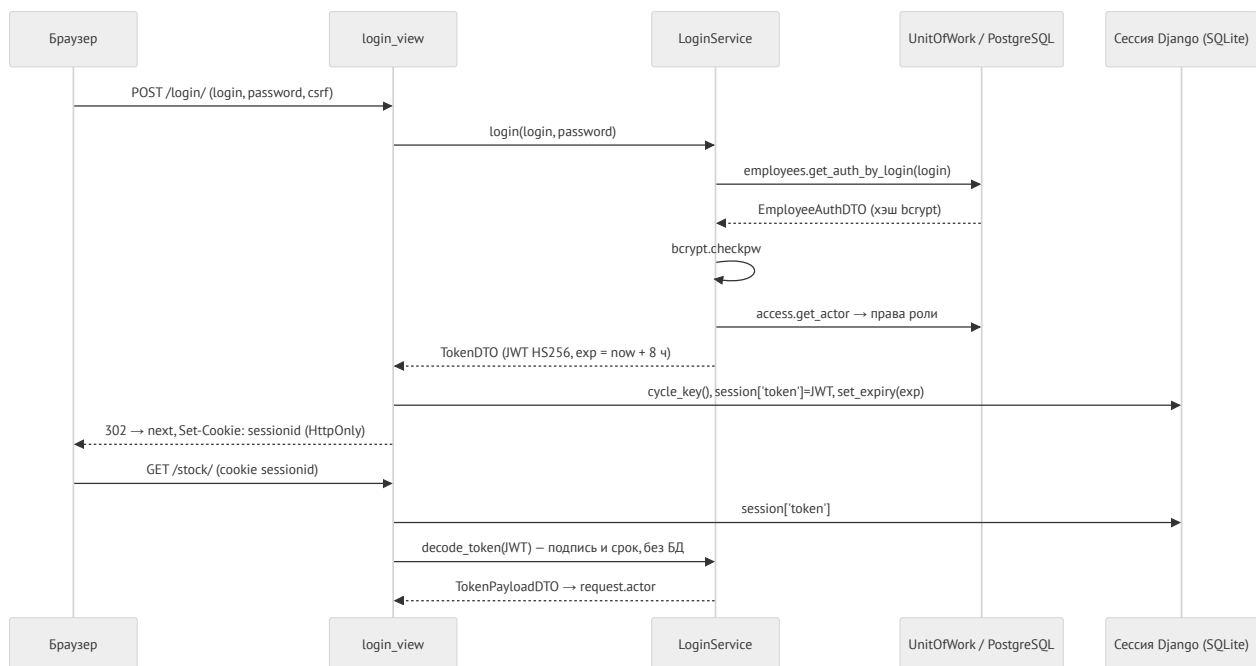


Схема «JWT внутри серверной сессии». Браузер получает не JWT, а обычный cookie `sessionid` (HttpOnly). Сам JWT лежит в сессии Django на сервере. На каждом запросе `employee_required` декодирует его: проверяет подпись и срок и раскладывает `employee_id`, `warehouse_id`, `role_name`, `permissions` в `request.actor`.

Почему так, а не чистая сессия или чистый JWT:

- **Против чистого JWT в Authorization.** Токен в `localStorage` доступен любому XSS-скрипту. Cookie `HttpOnly` из JS не прочитать.
- **Против чистой сессии.** `LoginService` уже выдаёт JWT, и его может использовать не-браузерный клиент. Сессия — лишь транспорт для браузера. Кроме того, `decode_token` не ходит в БД, поэтому проверка входа дешёвая.

Жизненный цикл:

- `sign_in` вызывает `request.session.cycle_key()`, то есть меняет идентификатор сессии при входе. Это защита от *session fixation*.
- `set_expiry(token.expires_at)` — сессия умирает вместе с токеном. Плюс `SESSION_COOKIE_AGE = JWT_EXPIRE_MINUTES*60`.
- `authenticate` при `AuthError` (истёк или подделан) вызывает `sign_out`, то есть `session.flush()`.
- `employee_required` различает «не входил» и «сессия истекла». Во втором случае показывается сообщение «Сессия истекла, войдите заново».

```
def employee_required(view):
    @wraps(view)
    def wrapper(request, *args, **kwargs):
        had_token = SESSION_TOKEN in request.session
        if authenticate(request) is not None:
            return view(request, *args, **kwargs)
        if had_token:
            messages.error(request, 'Сессия истекла, войдите заново')
        if request.method != 'GET':
            return redirect('login') # POST повторить после входа
        # нельзя — без next
        query = urlencode({REDIRECT_FIELD_NAME: request.get_full_path()})
        return redirect(f"{reverse('login')}?{query}")
    return wrapper
```

8.2.1 LoginService

```
def login(self, login: str, password: str) -> TokenDTO:
    login = (login or "").strip()
    if not login or not password:
        raise AuthError(INVALID_CREDENTIALS)
    with self.uow as uow:
        auth = uow.employees.get_auth_by_login(login) # удалённые не возвращаются
        # Одинаковый текст для «нет логина» и «неверный пароль»
        if auth is None or not self._password_matches(password, auth.password_hash):
            raise AuthError(INVALID_CREDENTIALS)
        actor = self.access.get_actor(uow, auth.employee.id)
        now = datetime.now(timezone.utc)
        payload = {"sub": str(actor.employee.id), "warehouse_id": ..., "role_name": ...,
                  "permissions": sorted(actor.permissions), "iat": now, "exp": now +
self._expire}
        token = jwt.encode(payload, self._secret_key, algorithm="HS256")
        ...
```

- `decode_token` жёстко фиксирует `algorithms=["HS256"]` (защита от атаки `alg=none`) и требует `sub` и `exp`.
- `_GENERATED_SECRET` — запасной ключ на случай, если секрет не передан. В веб-слое он не используется, потому что `settings.JWT_SECRET` передаётся всегда.
- Битый хэш в БД (`ValueError` от `bcrypt`) считается неверным паролем, а не ошибкой 500.

Тайминг-атака. Если логина нет, `bcrypt.checkpw` не вызывается, и ответ приходит примерно на 250 мс быстрее. Одинаковый текст ошибки не спасает от перебора логинов по времени ответа. Исправление — сверять пароль с заранее вычисленным фиктивным хэшем, когда пользователь не найден.

8.3 Permissions и проверка склада (`AccessService`)

```
class AccessService(AbstractAccessService):
    def get_actor(self, uow, employee_id) -> ActorDTO:
        employee = uow.employees.get_by_id(employee_id)           # заблокированный → None
        if employee is None:
            raise NotFoundError(f"Сотрудник №{employee_id} не найден или заблокирован")
        return ActorDTO(employee=employee,
            permissions=frozenset(uow.employees.get_permissions(employee_id)))

    def require_permission(self, actor, permission):
        if permission not in actor.permissions:
            raise AccessDeniedError("Недостаточно прав для этого действия")

    def require_warehouse(self, actor, warehouse_id):
        if self.is_admin(actor):                                  # администратор — все
            return
        if actor.employee.warehouse_id != warehouse_id:
            raise AccessDeniedError("Доступ запрещён. Вы работаете на другом складе")

    def is_admin(self, actor):
        return PermissionName.EMPLOYEE_MANAGE in actor.permissions
```

Важная деталь. Права в JWT (`request.actor.permissions`) используются **только для отображения** (скрыть кнопку). Каждый метод сервиса заново вызывает `get_actor` и берёт права из БД. Поэтому блокировка сотрудника или смена роли действуют сразу, хотя токен живёт 8 часов: страница откроется, а любое действие вернёт отказ.

Порядок проверок «кто → куда → что» (docstring `ShipmentReceiptService`): сначала право, затем склад, в последнюю очередь статус. Иначе по тексту ошибки «Этап в статусе Отправлено» сотрудник чужого склада узнал бы состояние чужого груза.

Матрица видимости маршрутов (`RouteQueryService`):

Права сотрудника	Что видит
<code>shipment:view_all</code> или <code>employee:manage</code>	Все этапы
<code>shipment:create</code> / <code>:dispatch</code> / <code>:accept</code>	Этапы, уходящие со своего склада или приходящие на него
Нет складских прав (водитель)	Этапы, где он водитель. В карточке — ещё и те, где он приёмщик

8.4 Декораторы проверки прав

Декораторов «на право» (как `@permission_required`) нет, и это сделано намеренно: право проверяется один раз, в сервисе. `can(request, permission)` из `web/auth.py` нужен только для условной отрисовки кнопок во `view`.

8.5 Session management

Параметр	Значение	Смысл
<code>SESSION_ENGINE</code>	по умолчанию <code>db</code>	Сессии в таблице <code>django_session</code> в SQLite
<code>SESSION_COOKIE_AGE</code>	<code>JWT_EXPIRE_MINUTES * 60 = 28 800</code> с	Кука живёт не дольше токена
<code>SESSION_COOKIE_HTTPONLY</code>	<code>True</code>	Недоступна из JS
<code>SESSION_COOKIE_SECURE</code>	не задан (<code>False</code>)	На production нужен <code>True</code> при HTTPS
Содержимое	<code>token</code> , <code>employee_name</code> , <code>warehouse_title</code>	JWT и данные для шапки

Для нескольких экземпляров приложения SQLite-сессии не подойдут: у каждого сервера будет своя база. Варианты: `SESSION_ENGINE = 'django.contrib.sessions.backends.signed_cookies'` (данных мало, а токен и так подписан) или `cache` с Redis.

9 Django REST Framework

9.1 Статус

DRF **не используется**, его нет в `requirements.txt`. Роль, которую обычно играет DRF, распределена по проекту так:

Задача DRF	Чем решена в Warehouse
Serializers (вход)	<code>_json_body()</code> + ручной разбор в view + нормализация в сервисе (<code>_to_quantity</code> , <code>_check_document</code>)
Serializers (выход)	<code>dataclasses.asdict(dto)</code> + <code>DjangoJSONEncoder</code>
ViewSets / Routers	Шесть классов <code>View</code> и ручные <code>path()</code>
Authentication classes	<code>api_employee_required</code> (сессия + JWT)
Permission classes	<code>AccessService</code> внутри BLL
Обработчик исключений	<code>SQLAlchemyAndBusinessErrorMiddleware.process_exception</code>
Throttling, фильтрация, пагинация	Нет
OpenAPI/Swagger	Нет

9.2 Сериализация на DTO

```
@dataclass(frozen=True, slots=True)
class StageDTO:
    id: int
    ...
    from_warehouse: WarehouseDTO      # вложенный DTO → вложенный JSON-объект
    items: tuple[StageItemDTO, ...] = ()  # tuple → JSON-массив
```

`asdict()` рекурсивно превращает вложенные `dataclass` в словари, а `DjangoJSONEncoder` сериализует `Decimal` строкой ("10.500", без потери точности), `date` и `datetime` — в ISO 8601. Это и есть «выходной сериализатор». Недостаток — наружу уходят все поля DTO. Например, `/api/auth/login/` отдаёт `EmployeeDTO` целиком, включая `role_id` и `is_deleted`.

9.3 Когда стоит перейти на DRF или Django Ninja

Если API станет основным каналом (мобильное приложение для водителей, интеграция с 1C), понадобятся схема OpenAPI, throttling и версионирование. Логичные варианты:

- **Django Ninja** — схемы на Pydantic (Pydantic и схемы из `common/schemas` уже есть), автоматический OpenAPI, стиль, близкий к FastAPI, с которого начинался проект;
- **DRF** — `Serializer` поверх `dataclass` (`rest_framework_dataclasses`), `APIView`, `SessionAuthentication`, `UserRateThrottle`, `drf-spectacular` для OpenAPI.

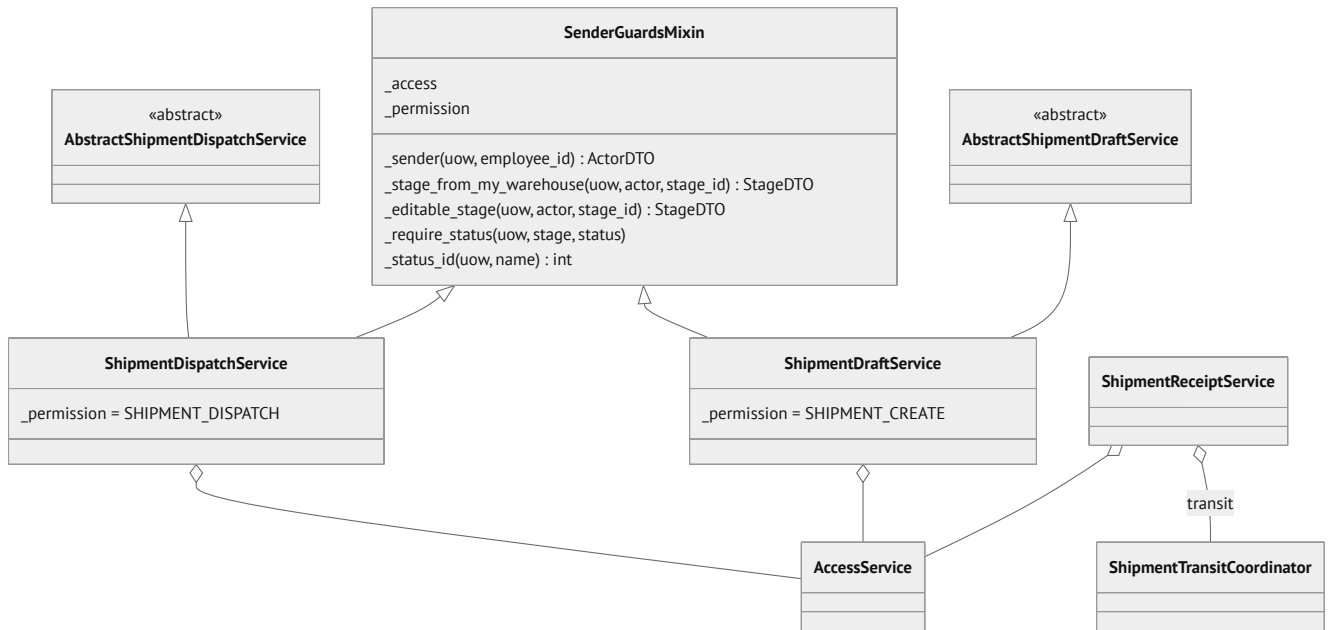
При любом варианте сервисы BLL не меняются: меняется только тонкая обёртка в `web/views/api.py`.

10 Бизнес-логика и сервисный слой

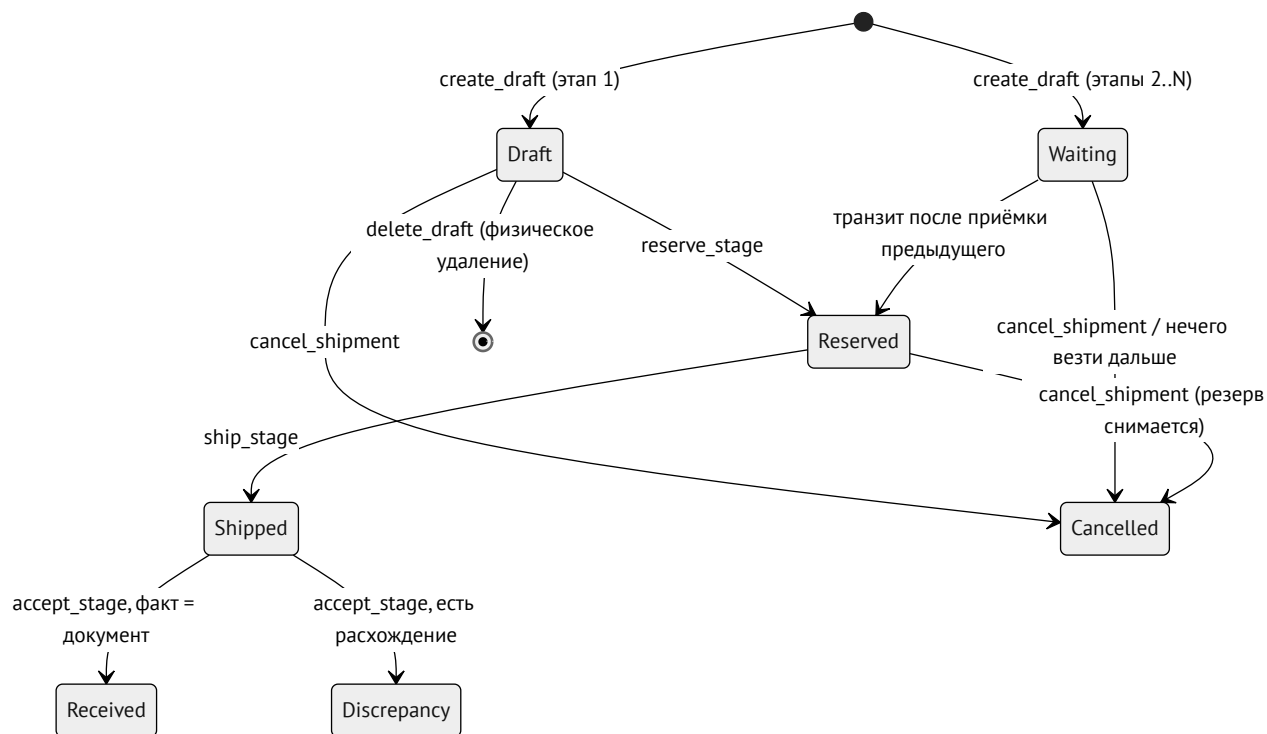
10.1 Состав BLL

Для каждого сервиса есть **интерфейс** (абстрактный класс, ABC) в `bll/interfaces/` и **реализация** в `bll/services/`. В docstring интерфейсов описан контракт: какие исключения и в каких случаях бросает метод.

Сервис	Право	Открывает транзакцию	Ответственность
AccessService	—	нет (получает uow)	Кто пришёл и что ему можно
LoginService	—	да	Вход, выпуск и проверка JWT
EmployeeService	employee:manage	да	Регистрация, справочники для формы, склад сотрудника
ShipmentDraftService	shipment:create	да	Черновик: маршрут, товары, водитель, документы
ShipmentDispatchService	shipment:dispatch / shipment:cancel	да	Резерв, отправка, отмена
ShipmentReceiptService	shipment:accept	да	Входящие, ввод факта, приёмка
ShipmentTransitCoordinator	—	нет (получает uow)	Что происходит после приёмки этапа
RouteQueryService	по матрице видимости	да	Чтение маршрутов, перевозок, документов

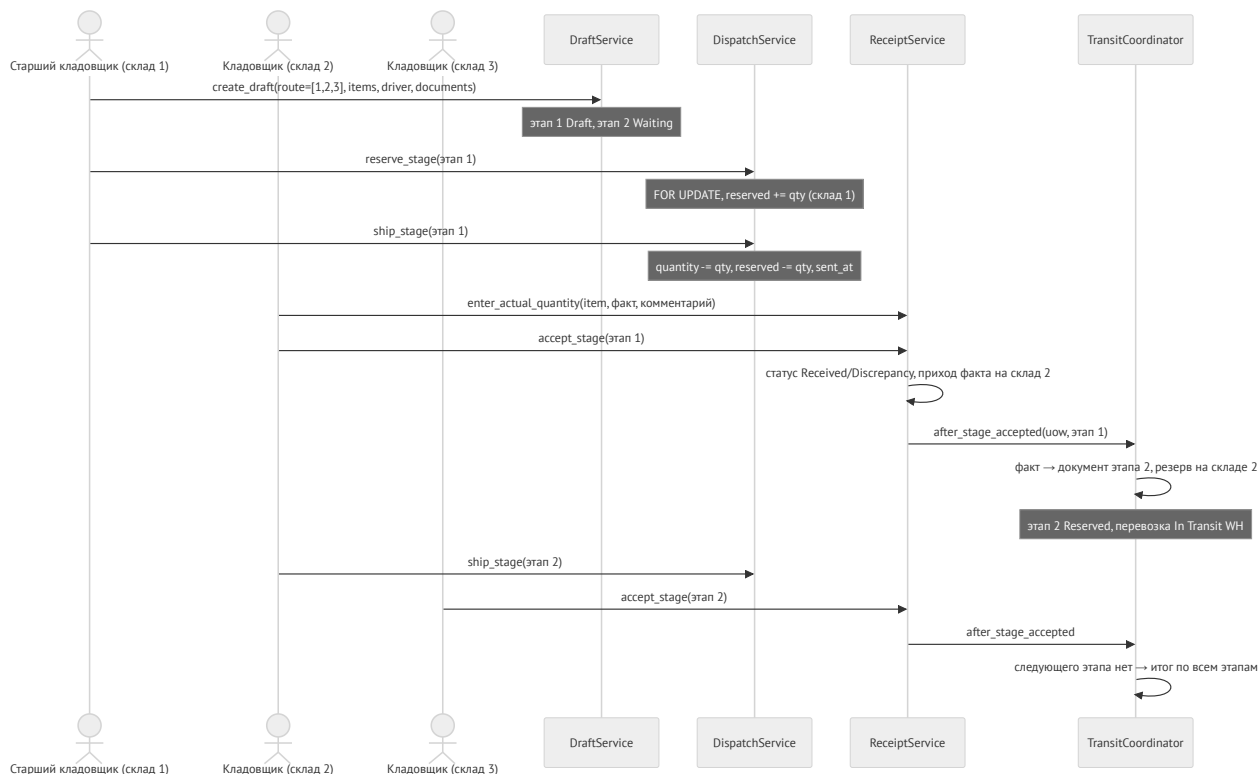


10.2 Жизненный цикл перевозки



Статусы **перевозки** меняются параллельно: Draft → Reserved → Shipped → In Transit WH → ... → Received | Discrepancy | Cancelled.

10.3 Процесс «перевозка через транзитный склад» целиком



10.4 ShipmentDraftService — черновик

10.4.1 create_draft

Порядок проверок:

1. Нормализация вне транзакции: `_to_quantity` для каждой позиции (`Decimal`, конечное, > 0) и `_check_document` (имя и путь непусты, размер ≥ 0).
2. `_sender` — есть ли `shipment:create`.
3. В маршруте не меньше двух складов; первый склад — склад сотрудника (`require_warehouse`).
4. `planned_date` не в прошлом.
5. Соседние склады маршрута различаются. `[1, 2, 1]` допустим: это возврат.
6. Все склады существуют и не удалены.
7. Товары не повторяются; свободного остатка хватает (**предварительно, без блокировки**).
8. Водитель существует, имеет роль «Водитель» и работает на складе отправления.

Затем одним вызовом `uow.shipments.create(...)` создаются перевозка, все этапы и позиции первого этапа (каскад `relationship`), и прикрепляются документы к этапу 1.

```
stages = [NewStage(from_warehouse_id=from_id, to_warehouse_id=to_id,
                  items=tuple(items) if order == 0 else (),          # товары — только в
этап 1
                  driver_id=driver_id if order == 0 else None)
          for order, (from_id, to_id) in enumerate(zip(route, route[1:]))] # [1, 2, 3] →
(1, 2), (2, 3)
```

Почему товары только в первом этапе. Что поедет дальше, определяет приёмка: при недостатке на складе 2 дальше едет факт, а не план. Ручное заполнение этапов 2..N противоречило бы этому правилу. Поэтому `_editable_stage` разрешает правку только этапа с `stage_order == 1` в статусе «Черновик».

10.4.2 Остальные методы

Метод	Особенность
<code>add_item</code>	Если товар уже есть в этапе — заменяет количество (<code>set_document_quantity</code>)
<code>remove_item</code>	Проверяет, что позиция принадлежит редактируемому этапу
<code>assign_driver</code>	Разрешено в Draft/Waiting/Reserved; <code>None</code> — снять водителя
<code>delete_draft</code>	Только если перевозка в статусе «Черновик»; каскадное удаление
<code>attach_document</code> / <code>remove_document</code>	Только этап 1 в статусе «Черновик». Файл на диске — забота web
<code>list_warehouses</code> , <code>list_available_stock</code> , <code>list_drivers</code> , <code>list_outgoing</code>	Справочники для форм

10.5 ShipmentDispatchService — резерв, отправка, отмена

10.5.1 reserve_stage — ключевая конкурентная операция

```
def reserve_stage(self, employee_id, stage_id):
    with self._uow_factory() as uow:
        actor = self._sender(uow, employee_id) # кто: shipment:dispatch
        stage = self._editable_stage(uow, actor, stage_id) # куда/что: свой склад,
        этап 1, Draft
        if not stage.items:
            raise ValidationError("В этапе нет ни одной позиции")
        if uow.stage_documents.count_by_stage(stage_id) == 0:
            raise ValidationError("Прикрепите к этапу хотя бы один документ: без него
резерв невозможен")

        warehouse_id = stage.from_warehouse.id
        # SELECT ... FOR UPDATE: строки остатков заблокированы до COMMIT
        stock = uow.stock.get_many(warehouse_id, (i.product_id for i in stage.items),
for_update=True)
        for item in stage.items:
            row = stock.get(item.product_id)
            available = row.available if row is not None else Decimal(0)
            if available < item.document_quantity:
                raise InvalidStatusError(f"На складе свободно {available} ...") # →
rollback всего резерва
            uow.stock.change(warehouse_id=warehouse_id, product_id=item.product_id,
reserved_delta=item.document_quantity) # reserved +=
qty

        reserved_id = self._status_id(uow, StatusName.RESERVED)
        uow.stages.set_status(stage_id, reserved_id)
        uow.shipments.set_status(stage.shipment_id, reserved_id)
        return self._stage(uow, stage_id) # COMMIT при выходе из
with
```

Почему блокировка обязательна. В `add_item` остаток проверяется без блокировки, и это только подсказка пользователю. Два кладовщика могут параллельно собрать черновики на один и тот же последний мешок кабеля. `FOR UPDATE` в `reserve_stage` выстраивает их в очередь: второй дожждётся коммита первого, перечитает остаток (именно для этого `populate_existing`) и получит отказ.

10.5.2 ship_stage

Разрешён только из статуса «Зарезервировано». Списывает `quantity` и `reserved_quantity` на `document_quantity`, ставит `sent_at` и переводит этап и перевозку в «Отправлено». Если резерв не найден, это считается сбоем данных, и отправка отклоняется до того, как `СНЕСК` в БД уронит коммит. Для этапа 2+ проверка документов не выполняется: документом служит принятый факт.

10.5.3 cancel_shipment

Требует `shipment:cancel` (менеджер или администратор). Перевозку и **все** её этапы можно отменить только в статусах `Draft/Waiting/Reserved`, то есть пока ничего не уехало. Для зарезервированных этапов резерв возвращается (`_release_reserve`), затем всё переводится в

«Отменено».

Проверки склада в отмене нет: менеджер с `shipment:cancel` может отменить перевозку любого склада. Для роли «Менеджер» с `view_all` это соответствует смыслу. Если право `cancel` когда-нибудь получит складская роль, потребуется `require_warehouse`.

10.6 ShipmentReceiptService — приёмка

Метод	Логика
<code>get_incoming</code>	Этапы «Отправлено», едущие на склад сотрудника, с позициями
<code>enter_actual_quantity</code>	Факт ≥ 0 ; при расхождении с документом обязателен комментарий
<code>check_discrepancies</code>	Позиции без факта или с расхождением (только чтение)
<code>accept_stage</code>	Все факты заполнены $\rightarrow$ статус Received/Discrepancy, <code>received_at</code> , <code>acceptor_id</code> $\rightarrow$ приход на склад $\rightarrow$ транзит

Порядок внутри `accept_stage` задокументирован комментариями и важен:

1. **Сначала финальный статус этапа.** Координатор по статусам этапов решает, чем закончилась вся перевозка.
2. **Затем приход** (`stock.add_quantity`, UPSERT). Следующий этап резервируется из этого остатка.
3. **Затем** `self._transit.after_stage_accepted(uow, stage_id)` — **в той же транзакции**. Если резерв на транзитном складе не удастся, откатится вся приёмка.

10.7 ShipmentTransitCoordinator — cross-docking

```
def after_stage_accepted(self, uow, stage_id):
    stage = uow.stages.get_by_id(stage_id, with_items=True)
    next_stage = uow.stages.get_by_order(stage.shipment_id, stage.stage_order + 1)
    if next_stage is None:
        self._finish_shipment(uow, stage)          # итог: Discrepancy, если хоть один
    # этап с расхождением
    else:
        self._move_to_next_stage(uow, stage, next_stage)
```

`_move_to_next_stage`:

- **едет то, что реально приняли:** позиции с `actual_quantity > 0` становятся документом следующего этапа (`_fill_next_stage` — UPSERT с учётом `UNIQUE(stage_id, product_id)` и удалением лишнего);
- резервирует их на транзитном складе с `FOR UPDATE`;
- этап N+1 $\rightarrow$ «Зарезервировано», перевозка $\rightarrow$ «На транзитном складе»;
- если везти нечего (всё принято нулём), `_close_remaining_stages` отменяет оставшиеся этапы и перевозку.

Спорный момент. Когда весь груз потерян (факт = 0 по всем позициям), перевозка получает статус «Отменено». Сам принятый этап при этом остаётся «Принято с расхождениями». Для отчётности логичнее было бы «Принято с расхождениями» у всей перевозки: отмена обычно означает «не состоялась по решению человека».

10.8 RouteQueryService — чтение

Все методы только читают данные и проверяют видимость по матрице из раздела 8.3. `list_routes` для сотрудника без складских прав вернёт пустой список, а не 403: «водителю нечего показать» — нормальная ситуация. Этапы, где сотрудник приёмщик, `_can_view_stage` показывает, но `list_routes` их не находит (там только `list_for_driver`). Небольшая несогласованность.

10.9 EmployeeService

`register` проверяет данные до открытия транзакции: дешёвые проверки не должны занимать соединение. Внутри транзакции проверяются права, склад, роль и занятость логина, затем пароль хэшируется `hash_password` (bcrypt, `gensalt()` = 12 раундов). `get_warehouse` нужен `login_view` для названия склада в шапке.

10.10 Общий код: SenderGuardsMixin

Draft и Dispatch работают со складом **отправления** и делят одни и те же проверки. Вместо дублирования проверки вынесены в миксин (коммит разделил `dispatch` на 2 сервиса + добавил миксин (под вопросом)). Каждый класс задаёт своё `_permission`.

```
class ShipmentDraftService(SenderGuardsMixin, AbstractShipmentDraftService):
    _permission = PermissionName.SHIPMENT_CREATE
```

«Под вопросом» — справедливая оценка. Миксин неявно требует от класса атрибутов `_access` и `_permission`. Альтернатива — композиция: отдельный класс `SenderGuards(access, permission)`, который передаётся в конструктор. `_status_id` и `_status_ids` дублируются ещё в `ShipmentReceiptService`, `ShipmentTransitCoordinator` и `RouteQueryService`. Их стоит поднять в `StatusRepository` как `require_id(name)`.

10.11 Signals, management commands, фоновые задачи

- **Django signals не используются** и не нужны: сохранения идут в обход Django ORM. Реакции на события (транзит после приёмки) вызываются явно, что читается проще сигналов.
- **Management commands нет.** Их роль частично выполняет `src/main.py` — CLI-проверка слоёв (*smoke test*): подключение к БД, справочники, выборки, откат в конце. Его стоит превратить в `python manage.py check_layers`.

- **Фоновых задач** (Celery, RQ) нет, все операции синхронные и короткие. Кандидаты на будущее — генерация PDF-накладных, уведомления водителю, чистка «осиротевших» файлов документов.
-

11 Работа с базой данных

11.1 Выбор СУБД

- **PostgreSQL** — данные склада. Нужны строки с блокировками (`FOR UPDATE`), `CHECK`, точные `numeric`, `INSERT ... ON CONFLICT` (используется в `StockRepository.add_quantity` через `sqlalchemy.dialects.postgresql.insert`) и `timestampz`. На SQLite или MySQL этот код без правок не заработает.
- **SQLite** (`db.sqlite3`) — только сессии Django.

11.2 Подключение (`dal/database.py`)

```
DATABASE_URL = URL.create(drivename="postgresql+psycopg2", username=..., password=...,
host=..., port=..., database=...)
engine = create_engine(DATABASE_URL, pool_pre_ping=True, pool_size=10, max_overflow=20)
_session_factory = sessionmaker(bind=engine, autoflush=False, expire_on_commit=False)
scoped_session_factory = scoped_session(_session_factory)  # одна сессия на поток
```

- `URL.create` вместо f-строки корректно экранирует пароль со спецсимволами (`@`, `/`).
- `pool_pre_ping` проверяет соединение перед выдачей, чтобы не получить «server closed the connection» после простоя.
- `pool_size=10`, `max_overflow=20` задаются **на процесс**. При четырёх воркерах gunicorn это до 120 соединений, а `max_connections` PostgreSQL по умолчанию — 100.
- `expire_on_commit=False` — объекты не «протухают» после коммита. Маппинг в DTO и так происходит внутри транзакции, но это защищает от случайного обращения после коммита.
- `autoflush=False` — изменения уходят в БД только на явном `flush()` (в `_add`) или `commit`.

11.3 Unit of Work

```
class UnitOfWork:
    def __enter__(self):
        self.session = self._session_factory()  # scoped: та же сессия в этом
        потоке
        self.employees = EmployeeRepository(self.session)
        ...  # 12 репозитория на одной сессии
        return self

    def __exit__(self, exc_type, exc, tb):
        try:
            if exc_type is None: self.session.commit()
            else: self.session.rollback()
        finally:
            self._session_factory.remove()  # закрыть сессию, вернуть
            соединение в пул
```

Паттерн Unit of Work — все изменения внутри одного `with` фиксируются одной транзакцией или откатываются целиком. Каждый публичный метод сервиса — один `with`, а значит, одна транзакция.

Подводный камень `scoped_session`. Сессия общая для потока. Если внутри одного `with uow` вызвать метод **другого** сервиса, который откроет свой `with`, внутренний `__exit__` закоммитит и **закроет** сессию внешнего. Внешний код продолжит работать уже с новой сессией, и атомарность нарушится. Сейчас этого нет: вложенные вызовы (`AccessService`, `TransitCoordinator`) получают `uow` параметром. Правило нужно соблюдать и дальше: **сервис, открывающий транзакцию, нельзя вызывать из другой транзакции**.

`SQLAlchemyAndBusinessErrorMiddleware` дополнительно вызывает `scoped_session_factory.remove()` после каждого запроса. Это страховка от утечки сессии, если код обратился к репозиторию вне `with`.

11.4 Репозитории

`BaseRepository[ModelT]` — обобщённая основа:

```
def _get(self, entity_id): return self.session.get(self.model, entity_id,
populate_existing=True)
def _one(self, stmt): return
self.session.scalar(stmt.execution_options(populate_existing=True))
def _all(self, stmt): return
list(self.session.scalars(stmt.execution_options(populate_existing=True)))
def _add(self, entity): self.session.add(entity); self.session.flush(); return entity
# id cпазы
def _update(self, id, **v): return
self.session.execute(update(self.model).where(self.model.id == id).values(**v)).rowcount >
0
def _delete(self, id): return
self.session.execute(delete(self.model).where(self.model.id == id)).rowcount > 0
```

Почему `populate_existing`. `_update` выполняет bulk UPDATE в обход identity map (кэша объектов сессии). Без `populate_existing` повторное чтение в той же транзакции вернуло бы старый объект из кэша: например, `reserve_stage` вернул бы этап в статусе «Черновик». Опция заставляет перезаписать кэш свежими данными.

`_update` / `_delete` опираются на `self.model.id`, поэтому для `StockOnWarehouse` и `RolePermission` (составной ключ) не применимы. `StockRepository` поэтому пишет свои `update` и `insert`.

Сводка публичных методов:

Репозиторий	Чтение	Запись
<code>EmployeeRepository</code>	<code>get_by_id</code> , <code>get_auth_by_login</code> , <code>login_exists</code> , <code>list_by_warehouse</code> , <code>list_by_role</code> , <code>get_permissions</code> , <code>has_permission</code>	<code>create</code> , <code>set_password_hash</code> , <code>soft_delete</code>
<code>RoleRepository</code> / <code>StatusRepository</code> / <code>MeasurementRepository</code>	<code>get_id</code> , <code>get_name</code> , <code>list_all</code> → {id: name}	—
<code>WarehouseRepository</code>	<code>get_by_id</code> , <code>list_active</code>	<code>create</code> , <code>soft_delete</code>

Репозиторий	Чтение	Запись
ProductRepository	get_by_id, get_by_article, list_active	create, soft_delete
StockRepository	get, get_many (+ for_update), list_by_warehouse, list_available	change (дельты), add_quantity (UPSERT)
ShipmentRepository	get_by_id, get_status_id, list_by_creator	create (с этапами), set_status, set_planned_date, delete
ShipmentStageRepository	get_by_id, get_by_order, list_by_shipment, list_all, list_for_warehouse, list_incoming, list_outgoing, list_for_driver, any_with_status	create, set_status, assign_driver, delete
StageItemRepository	get_by_id, list_by_stage, count_unfilled	add, add_many, set_document_quantity, set_actual_quantity, delete
StageDocumentRepository	get_by_id, list_by_stage, count_by_stage	add, delete
HistoryRepository	—	log_operation (не используется)

11.5 Оптимизация запросов: аналоги `select_related` / `prefetch_related`

Django	SQLAlchemy	Где
<code>select_related('fk')</code>	<code>joinedload(rel)</code> — JOIN в том же запросе	статусы, склады, водитель этапа, единица измерения
<code>select_related</code> + фильтр по связи	<code>join(rel) + contains_eager(rel)</code>	<code>EmployeeRepository._select</code> , <code>StockRepository._select</code>
<code>prefetch_related('children')</code>	<code>selectinload(rel)</code> — второй запрос <code>WHERE id IN (...)</code>	этапы перевозки, позиции этапа

```

STAGE_OPTIONS = (
    joinedload(ShipmentStage.shipment).joinedload(Shipment.status),
    joinedload(ShipmentStage.status),
    joinedload(ShipmentStage.from_warehouse),
    joinedload(ShipmentStage.to_warehouse),
    joinedload(ShipmentStage.driver),
)
ITEMS_OPTION =
selectinload(ShipmentStage.items).joinedload(StageItem.product).joinedload(Product.measurement)

```

Коллекции (1:N) грузятся через `selectinload`, чтобы JOIN не размножал строки родителя. Ссылки N:1 — через `joinedload`. Константы переиспользуются в `ShipmentRepository`, поэтому маппер `to_shipment` не делает ни одного ленивого запроса (проблема N+1 исключена).

11.6 Индексы

Все FK проиндексированы: `ix_Employees_role_id`, `ix_Employees_warehouse_id`, `ix_Products_measurement_id`, `ix_StockOnWarehouse_product_id`, `ix Shipments_creator_id`, `ix Shipments_status_id`, `ix ShipmentStages_*` (status, from, to, driver, acceptor),

`ix_StageItems_product_id`, `ix_StageDocuments_*`. Составные `UNIQUE (shipment_id, stage_order)` и `(stage_id, product_id)` одновременно служат индексами для поиска этапов перевозки и позиций этапа. Отдельные индексы по `ShipmentStages.shipment_id` и `StageItems.stage_id` не нужны, потому что это ведущие столбцы уникальных индексов.

Возможное улучшение: частичный индекс для активных маршрутов — `CREATE INDEX ... ON "ShipmentStages"(to_warehouse_id) WHERE status_id = <Shipped>`. Его имеет смысл вводить только при заметных объёмах.

11.7 Транзакции и атомарность

- Одна операция пользователя — одна транзакция (`with self._uow_factory() as uow`).
- Уровень изоляции — `READ COMMITTED` (по умолчанию в PostgreSQL). Гонки за остатки закрывает `SELECT ... FOR UPDATE`.
- `StockRepository.change` меняет значение **атомарной дельтой** (`quantity = quantity + :delta`), а не «прочитал → посчитал → записал». Даже без блокировки `UPDATE` не потеряет чужое изменение.

Ошибка в комментарии о взаимоблокировках. `get_many` сортирует `product_ids` и пишет в комментарии: «гарантирует одинаковый порядок блокировок». Это не так: порядок значений внутри `IN (...)` не задаёт порядок, в котором PostgreSQL читает и блокирует строки. Для детерминированного порядка нужен `ORDER BY product_id` в самом запросе:

```
python stmt = self._select(warehouse_id, for_update).where(
    StockOnWarehouse.product_id.in_(unique_sorted_ids)
).order_by(StockOnWarehouse.product_id) # вот это фиксирует порядок блокировок
```

Без этого две транзакции, резервирующие товары {1, 2} и {2, 1}, могут поймать deadlock. PostgreSQL разорвёт его ошибкой, одна транзакция откатится, и пользователь увидит ошибку 500.

Ещё один устаревший комментарий: в docstring `StockRepository.change` сказано «В скрипте создания таблиц CHECK-ограничений на остатки нет». В актуальном `sql_fullset.txt` они есть: `quantity >= 0`, `reserved_quantity >= 0`, `reserved_quantity <= quantity`.

11.8 Кэширование запросов и database routers

Кэширования нет. Router не нужен, потому что Django ORM работает с одной базой (SQLite), а PostgreSQL обслуживает SQLAlchemy. Идея кэширования справочников описана в главе 16.

11.9 Схема и начальные данные

`sql_fullset.txt` выполняется одной транзакцией (`BEGIN ... COMMIT`): 13 таблиц, CHECK на непустые строки, индексы и начальные данные — 4 единицы измерения, 8 статусов, 6 прав, 5 ролей и связи, 3 склада, 1 сотрудник, 3 товара и остатки на складе 1. В конце `setval(pg_get_serial_sequence(...))`

сдвигает счётчики `IDENTITY` за максимальный вставленный `id`. Иначе первая вставка без `id` упала бы с `duplicate key`.

Проблема с тестовым пользователем. Комментарий в SQL обещает вход `ivan_sklad / Sklad2026`. Проверка `bcrypt.checkpw(b'Sklad2026', <хэш из скрипта>)` возвращает **False**: хэш не соответствует паролю. Войти этим пользователем нельзя. Нужно сгенерировать правильный хэш:

```
bash python -c "from warehouse.common.security import hash_password;
print(hash_password('Sklad2026'))"
```

12 Статические файлы и медиа

12.1 Настройки

```
STATIC_URL = 'static/'
STATIC_ROOT = BASE_DIR / 'staticfiles'      # сюда collectstatic собирает статику для веб-сервера
MEDIA_URL = 'media/'
MEDIA_ROOT = BASE_DIR / 'media'            # сюда default_storage кладёт документы этапов
MAX_UPLOAD_SIZE = 10 * 1024 * 1024         # собственная настройка проекта
DATA_UPLOAD_MAX_MEMORY_SIZE = MAX_UPLOAD_SIZE
```

STATICFILES_DIRS не задан: единственный источник — web/static/ (находит AppDirectoriesFinder). staticfiles/ и media/ исключены из git.

12.2 Сборка и раздача статики

При `DEBUG=1` статику раздаёт `runserver`. При `DEBUG=0` её никто не раздаёт: нет ни `WhiteNoise`, ни настроенного `nginx`. Минимальный путь к `production`:

```
MIDDLEWARE.insert(1, 'whitenoise.middleware.WhiteNoiseMiddleware') # сразу после SecurityMiddleware
STORAGES = {"default": {"BACKEND": "django.core.files.storage.FileSystemStorage"},
            "staticfiles": {"BACKEND": "whitenoise.storage.CompressedManifestStaticFilesStorage"}}}
```

и `python manage.py collectstatic --noinput` при деплое. Шрифты берутся с Google Fonts, поэтому интерфейс зависит от внешнего CDN. Для закрытой сети шрифты нужно положить в `static/`.

12.3 Пользовательские загрузки

Документы этапов сохраняются через `default_storage` в `media/stage_documents/<uuid>.<ext>`.

Маршрута для `MEDIA_URL` нет, и это правильно: документы нельзя отдавать всем подряд. Их выдаёт `document_open_view (/stages/<id>/documents/<id>/`, пока заглушка) после проверки видимости этапа:

```
@require_GET
@employee_required
@business_errors_as_http
def document_open_view(request, stage_id: int, document_id: int):
    documents = route_service().list_documents(request.actor.employee_id, stage_id) # проверка прав
    document = next((d for d in documents if d.id == document_id), None)
    if document is None:
        raise Http404('Документ не найден')
    return FileResponse(open_stage_file(document.storage_path),
```

```
as_attachment=False, filename=document.file_name,  
content_type=document.content_type or 'application/octet-stream')
```

На production с nginx большие файлы лучше отдавать через X-Accel-Redirect, а не силами Python. При переходе на S3 достаточно сменить `STORAGES["default"]` на `storages.backends.s3.S3Storage` (django-storages): код `uploads.py` не изменится.

12.4 Frontend-сборщики

Не используются. Webpack или Vite понадобятся, только если интерактивные экраны прототипа (поиск с подсказками, конструктор маршрута) переедут на JS-фреймворк. Для текущего объёма хватит серверного рендеринга и небольшого JS, либо htmx.

13 Тестирование

13.1 Структура

```
web/tests.py # 14 тестов Django TestCase (веб-слой)
test/bll/services/employee_service/
    test_employee_service.py # 6 тестов unittest (BLL)
test/conftest.py, test/api/, test/web/,
test/bll/services/shipment_service/ # пустые заготовки
```

Результат прогона при подготовке книги:

```
$ python manage.py test web
Ran 14 tests in 0.034s - OK

$ PYTHONPATH=src python -m unittest discover -s test -t .
Ran 6 tests in 0.339s - OK
```

13.2 Тесты веб-слоя (web/tests.py)

Идея: **проверить сайт без PostgreSQL**. `WebTestCase.setUp` подменяет провайдеры контейнера моками:

```
for provider, service in ((container.login_service, self.login),
                           (container.employee_service, self.employees)):
    provider.override(service) # dependency-injector: любой вызов
    # провайдера вернёт мок
    self.addCleanup(provider.reset_override)
```

Это работает для обоих стилей получения сервиса: и `services.login_service()`, и `Provide[Container.login_service]`. `TestCase` Django создаёт тестовую SQLite-базу для сессий.

Класс	Тест	Что гарантирует
LoginTests	<code>test_anonymous_is_redirected_to_login_with_next</code>	Аноним → <code>/login/?next=/stock/</code>
	<code>test_login_puts_token_in_session_and_shows_header</code>	Токен в сессии, ФИО и склад в шапке, кнопка «Новый сотрудник» у админа
	<code>test_wrong_password_keeps_login</code>	Текст ошибки и сохранённый логин
	<code>test_next_to_other_host_is_ignored</code>	Защита от open redirect
	<code>test_expired_token_sends_back_to_login</code>	Истёкший токен удаляется из сессии
	<code>test_logout_clears_session</code>	Выход чистит сессию
RegisterTests	<code>test_admin_registers_employee</code>	Форма → <code>NewEmployee</code> с правильными типами
	<code>test_passwords_must_match</code>	Ошибка формы, сервис не вызван

Класс	Тест	Что гарантирует
	<code>test_service_error_is_shown_over_form</code>	<code>ValidationError</code> сервиса видна над формой
	<code>test_not_admin_gets_403</code>	<code>AccessDeniedError</code> → 403, кнопка скрыта
<code>StubAndTagTests</code>	<code>test_every_stub_page_opens</code>	Все заглушки открываются
	<code>test_stub_button_redirects_back_with_message</code>	POST-заглушка возвращает на Referer
	<code>test_api_without_login_is_401</code>	API без входа → 401
	<code>test_tags</code>	<code>status_pill</code> , <code>qty</code> с локалью ru

13.3 Unit-тесты BLL

`EmployeeServiceTests` подменяет `UnitOfWork` объектом `MagicMock` (`uow.__enter__.return_value = uow`) и подставляет его через `uow_factory=lambda: self.uow`. Реальный `AccessService` при этом используется. Проверяется: пароль захеширован и проходит `verify_password`, имя обрезано, занятый логин, короткий пароль, неизвестная роль, отказ не-администратору, сортировка ролей.

Инъекция `uow_factory` через конструктор — прямая выгода архитектуры: сервис тестируется без БД и без Django.

13.4 Чего не хватает

- Тестов сервисов перевозок.** Каталог `shipment_service/` пуст, а это самая рискованная логика. Приоритеты: резерв при нехватке остатка, отказ на чужом складе, приёмка с расхождением без комментария, транзит с недостатчей, отмена с возвратом резерва.
- Интеграционных тестов DAL на настоящем PostgreSQL.** Моки не проверят `FOR UPDATE`, `ON CONFLICT`, `CHECK` и `populate_existing`. Рекомендуются `pytest + testcontainers[postgres]` или отдельная тестовая база, которую фикстура создаёт из `sql_fullset.txt` и оборачивает каждый тест в транзакцию с откатом.
- Фабрик данных.** `factory_boy` поддерживает `SQLAlchemy` (`SQLAlchemyModelFactory`) и сократит ручные `EmployeeDTO(...)`.
- Покрывтия.** `coverage run manage.py test web && coverage report`. `.coverage` и `htmlcov/` уже в `.gitignore`.
- Единого раннера.** Два набора запускаются по-разному. `pytest + pytest-django` с `pythonpath = src` в `pyproject.toml` соберут всё одной командой. В истории есть коммит `add pytest settings`, но файла настроек в репозитории нет.

Пример теста резерва, который стоит написать первым:

```
def test_reserve_fails_when_stock_is_not_enough(self):
    self.uow.stages.get_by_id.return_value = stage(items=(item(product_id=1,
qty=Decimal('10'))),)
    self.uow.statuses.get_id.side_effect = STATUS_IDS.get
    self.uow.stage_documents.count_by_stage.return_value = 1
    self.uow.stock.get_many.return_value = {1: stock(quantity=Decimal('5'),
reserved=Decimal('0'))}
    with self.assertRaisesRegex(InvalidStatusError, 'свободно 5'):
```

```
self.service.reserve_stage(SENIOR.id, stage_id=1)
self.uow.stock.change.assert_not_called()           # ни одна строка не
зарезервирована
```

13.5 CI/CD

Пайплайна нет. Минимальный GitHub Actions:

```
name: tests
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:17
        env: { POSTGRES_PASSWORD: postgres, POSTGRES_DB: Warehouse }
        ports: ["5432:5432"]
        options: --health-cmd pg_isready
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "3.13" }
      - run: pip install -r requirements.txt coverage
      - run: psql -h localhost -U postgres -d Warehouse -f sql_fullset.txt
        env: { PGPASSWORD: postgres }
      - run: coverage run manage.py test web && PYTHONPATH=src coverage run -a -m unittest
discover -s test -t .
        env: { DEBUG: "1", DB_PASSWORD: postgres }
      - run: coverage report
```

14 Конфигурация и окружение

14.1 Переменные окружения

Переменная	По умолчанию	Кто читает	Назначение
DB_HOST, DB_PORT, DB_NAME, DB_USER, DB_PASSWORD	localhost, 5432, Warehouse, postgres, —	dal/database.py	Подключение к PostgreSQL
DEBUG	0 (выключен)	settings.env_bool	Режим отладки: 1/true/yes/on
SECRET_KEY	при DEBUG — случайный	settings.env_secret	Подпись сессий и CSRF Django
JWT_SECRET	при DEBUG — случайный	settings.env_secret → LoginService	Подпись JWT
JWT_EXPIRE_MINUTES	480	settings	Срок токена и сессии
ALLOWED_HOSTS	пусто	settings	Список хостов через запятую

14.2 Разбор `env_secret`

```
def env_secret(name: str) -> str:
    """Секрет из окружения; без него в DEBUG выдумывается временный, иначе ошибка."""
    value = os.getenv(name, '').strip()
    if value:
        return value
    if DEBUG:
        return get_random_secret_key()      # dev: запускается сразу, но ключ новый на
каждый запуск
    raise ImproperlyConfigured(f'В .env не задан {name}')  # prod: лучше упасть, чем
работать с пустым ключом
```

Функция вызывает `DEBUG`, объявленный **ниже** её определения. Это работает, потому что тело функции выполняется при вызове, а вызов стоит уже после `DEBUG = env_bool('DEBUG')`.

Последствие случайного ключа в dev: после каждого перезапуска `runserver` (в том числе при автоперезагрузке после правки кода) все сессии и токены становятся недействительны, и нужно входить заново. Если это мешает, задайте `SECRET_KEY` и `JWT_SECRET` в `.env` и на своей машине. При нескольких процессах (gunicorn с `-w 4`) случайный `JWT_SECRET` будет у каждого свой, и токен, выданный одним воркером, отвергнут остальные. Отсюда и комментарий в `.env.example`: «один на все процессы сервера».

14.3 Окружения dev / staging / prod

Сейчас это один `settings.py`, а различия задаются через `.env`. Для текущего объёма это нормально. Рекомендуемый рост:

```
config/settings/
  base.py      # общее
  dev.py       # from .base import *; DEBUG = True; логирование SQL
  prod.py      # from .base import *; безопасные куки, HSTS, WhiteNoise, JSON-логи
```

`DJANGO_SETTINGS_MODULE=config.settings.prod` задаётся в окружении сервера. Альтернатива — остаться на одном файле и вынести prod-флаги в `.env` (`SECURE_SSL_REDIRECT=1` и т. п.). Это ближе к 12-factor и к текущему стилю проекта.

14.4 Best practices, которые уже соблюдены

- Секретов нет в git: `.env` в `.gitignore`, в `.env.example` только пустые значения.
- В production приложение не запустится без ключей: *fail fast*.
- Отсутствующие обязательные приложения (`auth`, `admin`) не подключены, чтобы не держать мёртвый код.
- `LANGUAGE_CODE='ru'`, `TIME_ZONE='Europe/Moscow'`, `USE_TZ=True`: в БД хранится UTC (`timestampz`), а отображается московское время.

14.5 Замечания

1. `dal/database.py` вызывает `load_dotenv()` **без пути**, то есть ищет `.env` от текущего каталога. Под Django переменные уже загружены из `BASE_DIR/.env`, и проблемы нет. Но `python src/main.py`, запущенный не из корня, может не найти `.env`.
2. Движок SQLAlchemy создаётся **при импорте** модуля. Любой импорт `warehouse.dal` требует установленного `psycopg2`, хотя подключения ещё нет: `create_engine` ленивый.
3. Проверка `planned_date < date.today()` использует системную дату сервера, а не `Europe/Moscow`. На сервере в UTC с 00:00 до 03:00 по Москве «сегодня» сервера — это ещё вчерашний день по Москве, поэтому пройдёт черновик со вчерашней плановой датой. На сервере восточнее Москвы наоборот: черновик на сегодняшнюю московскую дату будет отклонён. Правильнее `django.utils.timezone.localtime()` или передавать «сегодня» в сервис параметром.

15 Безопасность

15.1 Реализованные меры

Угроза	Защита	Где
Утечка паролей	bcrypt с солью, 12 раундов	common/security.py
Перебор логинов по тексту ошибки	Одинаковое «Неверный логин или пароль»	LoginService.login
Подделка токена	HS256, фиксированный список алгоритмов, обязательные sub и exp	LoginService.decode_token
Кража токена через XSS	Токен в серверной сессии, cookie HttpOnly	web/auth.py, settings
Session fixation	session.cycle_key() при входе	sign_in
CSRF	CsrfViewMiddleware, {% csrf_token %} во всех формах, выход только по POST	settings, шаблоны
XSS	Автоэранирование шаблонов, format_html в тегах	Django, warehouse_tags.py
SQL-инъекции	Только параметризованные запросы SQLAlchemy Core/ORM, ни одной склейки SQL	DAL
Clickjacking	XFrameOptionsMiddleware (DENY)	settings
Open redirect	url_has_allowed_host_and_scheme для next и Referer	web/views/auth.py, web/errors.py
Горизонтальный доступ к чужим складам	require_warehouse в каждой операции, порядок «кто → куда → что», лог попыток	BLL
Утечка состояния чужого груза	Проверка склада до проверки статуса	ShipmentReceiptService
Загрузка вредоносных файлов	Белый список расширений, лимит 10 МБ, случайное имя на диске, нет публичного MEDIA_URL	web/uploads.py
Устаревшие права	Права перечитываются из БД при каждом действии	AccessService.get_actor
Секреты в репозитории	.env в .gitignore, fail fast при пустом ключе в prod	settings

15.2 Найденные слабые места и рекомендации

- SQL-логи раскрывают данные.** setup_logging() включает sqlalchemy.engine на уровне INFO. На этом уровне SQLAlchemy пишет каждый запрос с параметрами, в том числе password_hash при регистрации, в консоль и в logs/warehouse.log. Это ещё и нагрузка на диск. На production нужен уровень WARNING. Если логи SQL нужны, включайте их только в dev и с create_engine(..., hide_parameters=True).
- Тайминг-атака на вход** (раздел 8.2.1). Нужно сравнивать пароль с фиктивным хэшем, когда логина нет.
- Нет ограничения попыток входа.** Лимит «5 попыток на логин и IP за 15 минут» можно сделать через django-axes, django-ratelimit или nginx limit_req.

4. `@csrf_exempt` на `/api/auth/login/` открывает *login CSRF*: сторонний сайт может незаметно залогинить сотрудника под чужой учётной записью. Лучше убрать исключение и выдавать CSRF-cookie через `ensure_csrf_cookie`.
5. **HTTPS-настройки не заданы.** Для production:

```
SECURE_PROXY_SSL_HEADER = ('HTTP_X_FORWARDED_PROTO', 'https') # если TLS terminates
nginx
SECURE_SSL_REDIRECT = True
SESSION_COOKIE_SECURE = True
CSRF_COOKIE_SECURE = True
SECURE_HSTS_SECONDS = 31536000
SECURE_HSTS_INCLUDE_SUBDOMAINS = True
SECURE_CONTENT_TYPE_NOSNIFF = True # по умолчанию True в современных Django
SECURE_REFERRER_POLICY = 'same-origin'
CSRF_TRUSTED_ORIGINS = ['https://warehouse.example.ru']
```

6. **Content Security Policy.** В `base.html` есть inline `<script>` и внешние шрифты. Для строгой CSP скрипт нужно вынести в файл, а шрифты раздавать локально. В Django 6 CSP поддерживается встроенным middleware (`django.middleware.csp.ContentSecurityPolicyMiddleware`, настройка `SECURE_CSP`).
7. **Нет журнала аудита.** `operation_history` не используется. Для складской системы вопрос «кто списал товар» критичен. Сейчас ответ есть только в текстовом логе (`logging.info` в сервисах) и в полях `creator_id`, `driver_id`, `acceptor_id`, `uploaded_by`.
8. **content_type загружаемого файла** берётся от клиента. При выдаче файла лучше определять тип по расширению из белого списка и отдавать `Content-Disposition: attachment` для всего, кроме PDF и изображений.
9. **Сотрудник с `employee:manage` может создать другого администратора.** Это ожидаемо, но стоит писать такие события в аудит.

15.3 Аудит безопасности

Рекомендуемые проверки:

```
python manage.py check --deploy # встроенный аудит настроек Django
pip-audit -r requirements.txt    # известные уязвимости зависимостей
bandit -r src web                # статический анализ Python
```

`check --deploy` при текущих настройках выдаст предупреждения W004 (HSTS), W008 (SSL redirect), W012 (`SESSION_COOKIE_SECURE`) и W016 (`CSRF_COOKIE_SECURE`). Все они закрываются блоком из п. 5.

16 Производительность и оптимизация

16.1 Узкие места

Место	Проблема	Оценка
StatusRepository.get_id	Отдельный SELECT на каждое имя статуса. accept_stage с транзитом делает 5–6 таких запросов	Мелко, но просто исправить
AccessService.get_actor	2 запроса (сотрудник + права) в начале каждой операции	Цена свежих прав. Приемлемо
bcrypt	≈ 0,25 с CPU на вход	Намеренно, но нужен лимит попыток (иначе это DoS-вектор)
SQL-логирование на INFO	Запись каждого запроса в файл и консоль	Отключить в prod
ShipmentRepository.select	selectinload этапов с полным STAGE_OPTIONS, в котором повторно join-ится уже загруженная перевозка	Лишний JOIN, некритично
list_all / list_for_warehouse	Без LIMIT: со временем вернут тысячи этапов	Нужна пагинация

16.2 Кэширование

Справочники (Statuses, Roles, Measurements) меняются только миграцией. Их можно загружать один раз на процесс:

```
class StatusRepository(BaseRepository[Status]):
    _cache: dict[str, int] | None = None  # на класс = на процесс

    def get_id(self, status_name: str) -> int | None:
        if StatusRepository._cache is None:
            StatusRepository._cache = {name: id_ for id_, name in self.list_all().items()}
        return StatusRepository._cache.get(status_name)
```

Кэш Django (CACHES) с Redis понадобится при появлении дашбордов: счётчики «требуется внимания» на главной можно кэшировать по складу на 30–60 секунд.

16.3 Оптимизация запросов

Уже сделано: joinedload/selectinload вместо N+1, exists() вместо count() для проверок (login_exists, has_permission, any_with_status), count(*) без загрузки строк (count_by_stage, count_unfilled), атомарные UPDATE с дельтой и UPSERT.

Что можно улучшить:

- списки без позиций (with_items=False) — уже так в list_routes / list_outgoing;
- ORDER BY product_id в get_many с FOR UPDATE (глава 11);

- для экранов со счётчиками — агрегирующий запрос `GROUP BY status_id` вместо загрузки всех этапов.

16.4 Асинхронность и CDN

Приложение синхронное (WSGI), и это правильно при `scoped_session` (сессия привязана к потоку). Переход на `async` потребует `AsyncSession` и `async_scoped_session` с `scopefunc=current_task` — это отдельный проект. Статику и шрифты на CDN стоит выносить, только если пользователи географически распределены. Внутренней системе склада достаточно `WhiteNoise` с `CompressedManifestStaticFilesStorage` (gzip/brotli и вечный кэш по хэшу имени).

17 Логирование и мониторинг

17.1 Конфигурация (`common/logger.py`)

```
def setup_logging():
    log_dir = "logs"  # относительно текущего каталога!
    os.makedirs(log_dir, exist_ok=True)
    log_format = "%(asctime)s [%(levelname)s] (%(filename)s:%(lineno)d) -> %(message)s"
    logging.basicConfig(level=logging.INFO, format=log_format, datefmt=..., handlers=
[logging.StreamHandler()])
    file_handler = RotatingFileHandler(os.path.join(log_dir, "warehouse.log"),
                                      maxBytes=5 * 1024 * 1024, backupCount=5,
encoding="utf-8")
    ...
    logging.getLogger("").addHandler(file_handler)
    logging.getLogger("sqlalchemy.engine").setLevel(logging.INFO)  # SQL в лог
```

Вызывается из `WebConfig.ready()` (Django) и из `src/main.py` (CLI). В Django поверх этого применяется и стандартный LOGGING фреймворка (он не задан, поэтому используется `DEFAULT_LOGGING`). Строка `[WARNING] (log.py:283) -> Unauthorized: /api/stock/` в выводе тестов — это логгер `django.request`, отформатированный корневым обработчиком проекта.

Замечания:

- Путь `logs` относителен текущему каталогу. Под `gunicorn` из `/` логи попытаются записаться в `/logs`. Лучше строить путь от `BASE_DIR` или писать только в `stdout` (12-factor), а ротацию оставить `journald` или `Docker`.
- `RotatingFileHandler` небезопасен при нескольких процессах: воркеры будут одновременно ротировать один файл. Альтернативы — `WatchedFileHandler` + `logrotate` или `stdout`.
- Все сервисы пишут через корневой логгер (`logging.info(...)`). Именованные логгеры `logger = logging.getLogger(__name__)` позволили бы настраивать уровни по модулям (например, `warehouse.bll` — `INFO`, `sqlalchemy` — `WARNING`).
- Идиоматичнее для Django: перенести конфигурацию в `settings.LOGGING` (`dictConfig`), а `setup_logging` оставить только для CLI.

17.2 Что и на каком уровне логируется

Уровень	События
INFO	Регистрация сотрудника, создание и удаление черновика, прикрепление документа, резерв, отправка, отмена, приёмка (с итоговым статусом). Также все SQL-запросы
WARNING	Попытка доступа к этапу чужого склада (<code>_sender_guards</code> , <code>ShipmentReceiptService</code>). HTTP 4xx от Django
ERROR/CRITICAL	<code>src/main.py</code> : провал проверок

Бизнес-события логируются с номерами сотрудника, этапа и склада. Этого достаточно, чтобы восстановить историю по тексту, но не для запросов «покажи всё, что делал сотрудник №5». Для этого и нужна `operation_history`.

17.3 Sentry, ELK, метрики

Не подключены. Рекомендации:

- **Sentry:** `sentry-sdk[django]` и `sentry_sdk.init(dsn=..., integrations=[DjangoIntegration(), SQLAlchemyIntegration()])`. Ошибки 500 и медленные транзакции попадут в Sentry вместе с контекстом запроса. `send_default_pii=False`, потому что в запросах есть логины.
 - **ELK / Loki:** писать JSON в stdout (`python-json-logger`) с полями `employee_id`, `stage_id`, `warehouse_id` (через `extra=`).
 - **Метрики:** `django-prometheus` (время ответа, коды), бизнес-счётчики — «этапов в пути», «расхождений за сутки». Дашборд в Grafana.
-

18 Развёртывание и DevOps

18.1 Текущее состояние

Docker, CI/CD и production-конфигурации нет. Проект запускается локально:

```
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
createdb Warehouse && psql -d Warehouse -f sql_fullset.txt # схема + начальные данные
python manage.py migrate # таблица сессий в SQLite
python manage.py runserver # .env создастся из
.env.example
python src/main.py # (опционально) smoke-
test слоёв
```

18.2 Docker и docker-compose (рекомендуемая схема)

```
FROM python:3.13-slim
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt gunicorn whitenoise
COPY . .
RUN DEBUG=1 python manage.py collectstatic --noinput
CMD ["gunicorn", "config.wsgi:application", "-b", "0.0.0.0:8000", "-w", "4", "--threads",
"1"]
```

```
services:
  db:
    image: postgres:17
    environment: { POSTGRES_DB: Warehouse, POSTGRES_PASSWORD: ${DB_PASSWORD} }
    volumes:
      - pgdata:/var/lib/postgresql/data
      - ./sql_fullset.txt:/docker-entrypoint-initdb.d/001_schema.sql:ro # только при
первом запуске
  web:
    build: .
    env_file: .env
    environment: { DB_HOST: db }
    volumes: [media:/app/media]
    depends_on: [db]
    ports: ["8000:8000"]
volumes: { pgdata: {}, media: {} }
```

Два нюанса. Во-первых, `db.sqlite3` с сессиями внутри контейнера пропадёт при пересоздании, и все выйдут из системы. Решение — сессии в signed cookies или Redis. Во-вторых, `manage.py` в контейнере не должен создавать `.env`: окружение приходит из `env_file`.

18.3 CI/CD

Пример тестового этапа приведён в главе 13. Деплой: сборка образа → push в registry → на сервере `docker compose pull && docker compose up -d` → выполнение миграций схемы.

18.4 Миграции в production

Пока схема меняется вручную, каждое изменение нужно оформлять отдельным SQL-файлом (`migrations/2026-10-01_add_x.sql`) и учитывать, какие файлы уже применены. Это и есть Alembic, только вручную. Переход на Alembic:

1. `alembic init migrations`; в `env.py` указать `target_metadata = Base.metadata` и URL из `warehouse.dal.database`.
2. Привести модели в соответствие с `sql_fullset.txt`: добавить недостающие `CHECK`, решить судьбу `operation_history`.
3. `alembic revision --autogenerate -m "baseline"`, проверить, что diff пуст, затем `alembic stamp head` на существующих базах.
4. Начальные данные вынести в отдельную ревизию (`op.bulk_insert`).

18.5 Backup и восстановление

```
pg_dump -Fc -d Warehouse -f warehouse_$(date +%F).dump      # ежедневно, с хранением N
копий
pg_restore -d Warehouse_restore warehouse_2026-10-01.dump  # проверка восстановления
- регулярно
```

Папку `media/` нужно бэкапить **синхронно** с базой: `StageDocuments.storage_path` без файла бесполезен. Для точки восстановления на момент времени — WAL-архивирование (`pgBackRest`, `WAL-G`).

18.6 Масштабирование

Приложение почти stateless. Горизонтальному масштабированию мешают три вещи:

1. **Сессии в локальном SQLite** → signed cookies или Redis.
2. **Файлы в локальной media/** → S3-совместимое хранилище.
3. **Пул соединений** — `pool_size` + `max_overflow` на каждый воркер. При росте поставить PgBouncer (transaction pooling) и уменьшить пул приложения.

Блокировки `FOR UPDATE` короткие (одна операция). Узким местом станет только очень «горячий» товар на одном складе.

19 Зависимости и требования

19.1 Пакеты

Пакет	Версия	Используется	Зачем	Альтернативы
SQLAlchemy	2.0.52	✓	ORM, блокировки, PostgreSQL-диалект	Django ORM (глава 20), SQLAlchemy
psycopg2-binary	2.9.13	✓	Драйвер PostgreSQL	psycopg 3 (postgresql+psycopg) — актуальная ветка, поддерживает асинх. -binary не рекомендуется для production-сборок
PyJWT	2.10.1	✓	JWT HS256	authlib, python-jose (менее поддерживается)
bcrypt	4.2.1	✓	Хэш паролей	argon2-cffi (Argon2id — рекомендация OWASP), passlib (заброшен)
dependency-injector	4.45.0	✓	DI-контейнер	Ручная фабрика в web/services.py без библиотеки, punq, lagom
Django	6.1.1	✓	Веб-слой	Flask, FastAPI (с которого начинали)
python-dotenv	1.2.2	✓	.env	django-environ, pydantic-settings
fastapi	0.115.8	✗	Наследие	Удалить
pydantic	2.10.6	✗ (только common/schemas, не импортируются)	Наследие	Удалить или использовать для валидации API
uvicorn	0.34.0	✗	Наследие	Удалить; для prod — gunicorn

Не хватает (dev): pytest, pytest-django, coverage, factory_boy, линтер (ruff). Для prod: gunicorn, whitenoise.

19.2 Версии Python и совместимость

- **Минимум — Python 3.12:** этого требует Django 6.x. Код использует StrEnum (3.11+) и X | None (3.10+).
- **Python 3.14:** при подготовке книги `pip install -r requirements.txt` на 3.14.7 **упал** на сборке pydantic-core (для pydantic 2.10.6) и dependency-injector 4.45.0: для 3.14 нет готовых колёс, а сборка из исходников не проходит. С dependency-injector 4.49.1 и pydantic 2.13.5 всё установилось, и тесты прошли.
- **Рекомендации:** зафиксировать версию Python в .python-version (файл уже в .gitignore, лучше закоммитить) или в pyproject.toml (requires-python = ">=3.12,<3.15"); обновить dependency-injector>=4.48; удалить pydantic и fastapi.

19.3 Разделение зависимостей

```
requirements/  
  base.txt    # SQLAlchemy, psycopg, PyJWT, bcrypt, dependency-injector, Django, python-  
dotenv  
  dev.txt     # -r base.txt; pytest, pytest-django, coverage, factory-boy, ruff  
  prod.txt    # -r base.txt; gunicorn, whitenoise, sentry-sdk
```

Или `pyproject.toml` с `[project.optional-dependencies]` и lock-файлом (`uv lock`, `pip-tools`).

20 Архитектурные решения и паттерны

20.1 Применённые паттерны

Паттерн	Реализация	Зачем
Layered Architecture	WEB → BLL → DAL, <code>common</code> сбоку	Бизнес-правила не зависят от HTTP и шаблонов
Service Layer	<code>*Service</code> в <code>bll/services</code>	Одна точка входа на операцию, транзакция на уровне сервиса
Repository	<code>*Repository</code> в <code>dal/repositories</code>	Все запросы в одном месте, наружу только DTO
Unit of Work	<code>UnitOfWork</code> (контекстный менеджер)	Атомарность, общая сессия у репозиториев
DTO	<code>common/dto</code>	Развязка слоёв, неизменяемые данные вне сессии
Data Mapper	<code>common/mappers</code> + SQLAlchemy	Отдельно модель, отдельно представление
Dependency Injection / IoC	<code>Container</code> , <code>providers.Factory</code> , <code>uow.provider</code>	Тестируемость (<code>override</code>), сборка графа в одном месте
Interface Segregation	ABC-интерфейсы на каждый сервис	Контракты (исключения, транзакции) в docstring
Mixin	<code>SenderGuardsMixin</code>	Общие проверки для склада-отправителя
Coordinator / Process Manager	<code>ShipmentTransitCoordinator</code>	Шаг процесса, не привязанный к действию пользователя
Decorator	<code>employee_required</code> , <code>business_errors_as_*</code>	Сквозная функциональность у view
Exception translation	Иерархия <code>BusinessError</code> с <code>status_code</code>	Одна ошибка BLL → HTTP, JSON или сообщение

Как устроена DI-связка `uow_factory=uow.provider`:

```
uow = providers.Factory(UnitOfWork, session_factory=session_factory)
draft_service = providers.Factory(ShipmentDraftService, uow_factory=uow.provider,
access=access_service)
```

`uow.provider` передаёт в сервис **сам провайдер** (вызываемый объект), а не готовый `UnitOfWork`. Поэтому каждый вызов `self._uow_factory()` создаёт новый UoW. В тестах вместо него передаётся `lambda: mock_uow`. `LoginService` получает `uow=uow`, то есть экземпляр. Это тоже работает, потому что UoW переиспользуем: каждый `with` открывает новую сессию. Но стиль несогласован.

20.2 Почему Django без Django ORM

Хронология объясняет выбор. Домен, DAL и BLL были написаны на SQLAlchemy под FastAPI (14–17.09). Django пришёл позже (23–25.09) ради шаблонов, форм, сессий и CSRF. Переписывать отлаженный DAL не стали.

Плюсы:

- BLL и DAL полностью независимы от веб-фреймворка: запускаются из CLI (`src/main.py`) и тестируются без Django;

- у SQLAlchemy мощнее контроль над SQL: `with_for_update(of=...)`, `contains_eager`, диалектный UPSERT, `populate_existing`;
- явные транзакции вместо `ATOMIC_REQUESTS`.

Минусы:

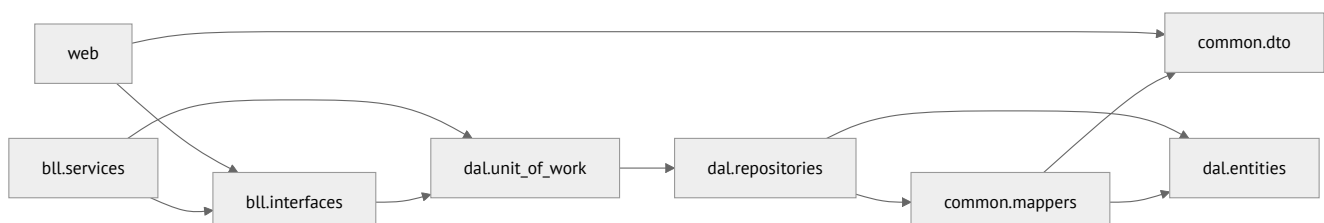
- нет Django admin: справочники, склады и товары пока правятся только через SQL;
- две базы (PostgreSQL + SQLite) и два механизма схемы;
- нет миграций (Django migrations не применимы, Alembic не подключён);
- `django.contrib.auth` и экосистема (`allauth`, `guardian`, `DRF-permissions`) недоступны без адаптеров;
- новым разработчикам на Django нужно освоить SQLAlchemy.

Альтернатива, от которой отказались (неявно): перенести модели в Django ORM. Тогда пришлось бы переписать 12 репозиториев на `QuerySet` (`select_for_update`, `F()`-выражения, `update_or_create`) и вернуться к `request.user`. Для проекта на этапе 0 вопрос открыт. Если админка важна, переход дешевле сделать сейчас, пока view почти пусты. Если важна независимость домена, текущий путь последовательнее.

20.3 Trade-offs

Решение	Выигрыш	Цена
Статусы по имени, а не по id	Устойчивость к расхождению id между базами	Лишний SELECT; имена со скобками «(Draft)» – часть контракта
Права перечисляются в каждой операции	Мгновенная блокировка и смена роли	2 запроса на операцию
JWT в серверной сессии	HttpOnly, дешёвая проверка	Два механизма там, где хватило бы одного
DTO вместо моделей в view	Нет ленивых загрузок, неизменяемость	Много кода-маппинга
Файлы вне транзакции	DAL не знает о файловой системе	Ручная компенсация, «осиротевшие» файлы
Товары вручную только в этапе 1	Дальше едет факт, а не план	Нельзя заранее спланировать разные грузы на разные этапы
Интерфейсы ABC на каждый сервис	Явный контракт	Одна реализация на интерфейс – формальность

20.4 Зависимости между пакетами



Заметно, что `common.mappers` зависит от `dal.entities`, то есть «общий» модуль знает о DAL. Логичнее перенести мапперы в `dal/mappers`. Кроме того, `bll.interfaces` импортирует **конкретный** `UnitOfWork`. Чистая архитектура предполагает протокол (`typing.Protocol`) `AbstractUnitOfWork` в BLL, который реализует DAL.

21 Расширение и развитие проекта

21.1 Как добавить новую функцию: сквозной пример

Задача: «Возврат черновика в работу» — снять резерв этапа 1 и вернуть его в «Черновик», чтобы поправить товары.

1. **Интерфейс** — `AbstractShipmentDispatchService.unreserve_stage(employee_id, stage_id) -> StageDTO` с описанием исключений.
2. **Реализация** — в `ShipmentDispatchService`:

```
def unreserve_stage(self, employee_id: int, stage_id: int) -> StageDTO:
    with self._uow_factory() as uow:
        actor = self._sender(uow, employee_id) # кто
        stage = self._stage_from_my_warehouse(uow, actor, stage_id) # куда
        if stage.stage_order != 1:
            raise InvalidStatusError("Вернуть в черновик можно только первый этап")
        self._require_status(uow, stage, StatusName.RESERVED) # что
        self._release_reserve(uow, stage) # FOR UPDATE +
        reserved -= qty
        draft_id = self._status_id(uow, StatusName.DRAFT)
        uow.stages.set_status(stage_id, draft_id)
        uow.shipments.set_status(stage.shipment_id, draft_id)
        logging.info("Сотрудник №%s вернул этап №%s в черновик", actor.employee.id,
            stage_id)
        return self._stage(uow, stage_id)
```

3. **Тест BLL** с моком `uow`.
4. **URL** — `path('stages/<int:stage_id>/unreserve/', detail.unreserve_view, name='stage_unreserve')`.
5. **View** — `@require_POST @employee_required @business_errors_as_messages`, одна строка вызова сервиса.
6. **Шаблон** — кнопка в форме с `{% csrf_token %}`, показывать при `'shipment:dispatch'` in `perms`.
7. **Тест view** — подмена `container.dispatch_service.override(mock)`.

Контейнер править не нужно: сервис уже зарегистрирован.

21.2 Точки расширения

Что добавить	Куда
Новая роль	<code>INSERT INTO "Roles" + RolePermissions</code> , константа в <code>RoleName</code>
Новое право	<code>Permissions + PermissionName</code> + проверка в сервисе
Новый статус	<code>Statuses + StatusName + STATUS_CLASSES</code> и CSS-класс
Новая сущность	<code>entity -> DTO -> mapper -> repository -> поле в UnitOfWork -> сервис -> провайдер в Container</code>

Что добавить	Куда
Другое хранилище файлов	<code>STORAGES["default"]</code>
Уведомления	Новый сервис, вызываемый из <code>accept_stage / ship_stage</code> после коммита (или <code>outbox</code> -таблица)

21.3 Рекомендации по рефакторингу (по приоритету)

Исправить сейчас (ошибки):

1. Хэш тестового пользователя в `sql_fullset.txt` (вход невозможен).
2. `ORDER BY product_id` в `StockRepository.get_many` при `for_update`.
3. SQL-логирование на INFO в `production`; утечка параметров.
4. Приведение `route` к `list[int]` в `CreateDraftView`.
5. `planned_date` сравнивать с `timezone.localdate()`.
6. Добавить `'л'` в `FRACTIONAL_UNITS`.
7. Устаревший docstring `StockRepository.change` о CHECK.

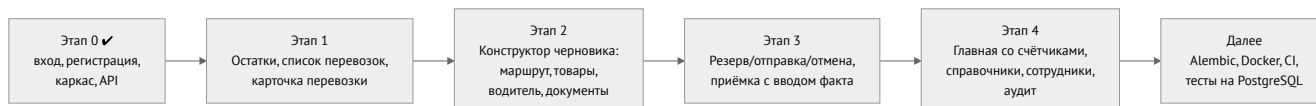
Почистить (мёртвый код):

8. `common/schemas/*`, `common/mappers/auth_mappers.py`, `common/mappers/shipments_statistics.py`, `fastapi`, `uvicorn`, `pydantic`.
9. `operation_history`: либо внедрить (таблица в SQL, запись в сервисах), либо удалить модель и репозиторий.
10. `erd.script`: удалить или сгенерировать заново из актуальной схемы; единым источником оставить Alembic.
11. Пустые `web/models.py`, `test/conftest.py`, `test/api`, `test/web`: удалить или наполнить.

Улучшить структуру:

12. Один способ получать сервисы во `view` (`web/services.py`).
13. `_status_id` → `StatusRepository.require_id` с кэшем.
14. Мапперы → `dal/mappers`, протокол `AbstractUnitOfWork` в BLL.
15. `pyproject.toml`: `src-layout` как пакет, настройки `pytest` и `ruff`, `requires-python`.
16. Документация: README с запуском (глава 18.1).

21.4 Roadmap (из прототипа и заглушек)



Раздел «Справочники» и «Сотрудники» (список и блокировка) есть в прототипе, но для них ещё нет сервисов: `soft_delete` в репозиториях есть, в BLL — нет.

Приложение А. Глоссарий

Термин	Определение
Перевозка (Shipment)	Перемещение товара по маршруту из ≥ 2 складов; «шапка» с общим статусом
Этап / путь (Stage)	Отрезок маршрута «склад А → склад В» со своим статусом, водителем, товарами и документами
Позиция (StageItem)	Товар в этапе: количество по документу и по факту
Остаток / резерв	<code>quantity</code> — сколько лежит; <code>reserved_quantity</code> — сколько обещано этапам; свободно = разность
Факт	Количество, реально принятое на складе назначения
Расхождение (Discrepancy)	Факт $\neq$ документ; требует комментария
Транзитный склад	Промежуточный склад маршрута, где груз принимают и отправляют дальше
Cross-docking	Перевалка груза через склад без длительного хранения
Мягкое удаление	Пометка <code>is_deleted = true</code> вместо <code>DELETE</code>
Actor	Сотрудник, выполняющий действие, вместе с его правами (ActorDTO)
BLL / DAL	Business Logic Layer / Data Access Layer
DTO	Data Transfer Object — неизменяемый объект для передачи данных между слоями
UoW	Unit of Work — одна транзакция и все репозитории на ней
DI / IoC	Dependency Injection / Inversion of Control — зависимости передаются извне
RBAC	Role-Based Access Control — права через роли
JWT	JSON Web Token — подписанный токен с полями <code>sub</code> , <code>exp</code> и др.
HS256	HMAC-SHA256, симметричная подпись JWT
bcrypt	Адаптивная хэш-функция паролей с солью
CSRF	Cross-Site Request Forgery — подделка запроса от имени пользователя
XSS	Cross-Site Scripting — внедрение скрипта в страницу
PRG	Post/Redirect/Get — редирект после успешного POST
N+1	Один запрос за списком + по запросу на каждый элемент
FBV / CBV	Function-/Class-Based View
FOR UPDATE	Блокировка выбранных строк до конца транзакции
UPSERT	<code>INSERT ... ON CONFLICT DO UPDATE</code>
12-factor	Методология: конфигурация в окружении, логи в <code>stdout</code> и т. д.

Приложение В. FAQ и Troubleshooting

ImproperlyConfigured: В `.env` не задан `SECRET_KEY`. Запуск с `DEBUG=0` без ключей. Задайте `SECRET_KEY` и `JWT_SECRET` (`python -c "import secrets; print(secrets.token_urlsafe(64))"`) или `DEBUG=1` для разработки.

ModuleNotFoundError: No module named 'warehouse' / 'container'. Код запущен в обход `manage.py` / `settings.py`, и `src` не попал в `sys.path`. Запускайте через `manage.py`, либо `PYTHONPATH=src`, либо установите пакет (`pip install -e .`).

Не удаётся войти под `ivan_sklad` / `Sklad2026`. Хэш в `sql_fullset.txt` не соответствует паролю (глава 11.9). Обновите `password_hash` хэшем от `hash_password('Sklad2026')`.

После каждой правки кода «Сессия истекла, войдите заново». В dev без `SECRET_KEY` / `JWT_SECRET` ключи генерируются заново при каждом перезапуске. Пропишите их в `.env`.

no such table: django_session. Не выполнен `python manage.py migrate`.

psycopg2.OperationalError: connection refused. PostgreSQL не запущен или неверны `DB_*`. Проверка: `python src/main.py` выведет `[FAIL]` Подключение к БД.

API отвечает 403 «CSRF verification failed» на POST. Нужен заголовок `X-CSRFToken` со значением cookie `csrftoken` (раздел 5.4).

`pip install` падает на `pydantic-core` / `dependency-injector` (Python 3.14). Обновите `dependency-injector` >= 4.48 и удалите `pydantic/fastapi` (глава 19.2) или используйте Python 3.12–3.13.

«В справочнике нет статуса «...»». Имена статусов в БД должны совпадать с `StatusName` символ в символ, включая часть в скобках.

Резерв: «Прикрепите к этапу хотя бы один документ». Это бизнес-правило: без документа этап не резервируется.

Приёмка: «Факт расходится с документом — заполните комментарий». Комментарий обязателен при любом отличии факта от документа, включая недостачу до нуля.

Логи огромные. Уровень `sqlalchemy.engine` = INFO в `setup_logging`. Поднимите до WARNING.

Страница 404/403 показывает отладочную информацию Django. Включён `DEBUG=1`. Кастомные `404.html` / `403.html` используются только при `DEBUG=0`.

Приложение С. Ссылки и ресурсы

Django

- Документация Django 6.1 — <https://docs.djangoproject.com/en/6.1/>
- Views и декораторы — <https://docs.djangoproject.com/en/6.1/topics/http/decorators/>
- Сессии — <https://docs.djangoproject.com/en/6.1/topics/http/sessions/>
- CSRF — <https://docs.djangoproject.com/en/6.1/ref/csrf/>
- Custom template tags — <https://docs.djangoproject.com/en/6.1/howto/custom-template-tags/>
- Deployment checklist — <https://docs.djangoproject.com/en/6.1/howto/deployment/checklist/>
- Статика — <https://docs.djangoproject.com/en/6.1/howto/static-files/>

SQLAlchemy и PostgreSQL

- SQLAlchemy 2.0 ORM — <https://docs.sqlalchemy.org/en/20/orm/>
- Relationship loading (joinedload/selectinload) — <https://docs.sqlalchemy.org/en/20/orm/queryguide/relationships.html>
- Contextual sessions (scoped_session) — <https://docs.sqlalchemy.org/en/20/orm/contextual.html>
- PostgreSQL: явные блокировки — <https://www.postgresql.org/docs/current/explicit-locking.html>
- PostgreSQL: INSERT ON CONFLICT — <https://www.postgresql.org/docs/current/sql-insert.html>
- Alembic — <https://alembic.sqlalchemy.org/>

Безопасность и прочее

- PyJWT — <https://pyjwt.readthedocs.io/>
- OWASP Password Storage Cheat Sheet — https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
- OWASP Authentication Cheat Sheet — https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html
- dependency-injector — <https://python-dependency-injector.ets-labs.org/>
- The Twelve-Factor App — <https://12factor.net/ru/>
- Martin Fowler, «Patterns of Enterprise Application Architecture» — Unit of Work, Repository, Service Layer, Data Mapper
- «Architecture Patterns with Python» (Percival, Gregory) — <https://www.cosmicpython.com/>

Инструменты разработчика

- `ruff` (линтер и форматтер), `pytest` + `pytest-django`, `coverage`, `pip-audit`, `bandit`
- pgAdmin 4 (ERD Tool), dbdiagram.io (визуальная схема [documentation/Untitled.png](#))
- Mermaid Live Editor — <https://mermaid.live/> (редактирование диаграмм этой книги)